AWS Cloud Practitioner Essentials
Niveau Fondamental

www.inow-formation.fr

Version 1 / Novembre 2025 / Vincent Leclerc

Interactions avec les services AWS

Console

AWS CLI

AWS SDK

Interactions avec les services AWS

Toutes les interactions avec les services sont alimentées par des API. Vous pouvez accéder à ces API à l'aide de trois méthodes principales : la console de gestion AWS, l'interface de ligne de commande AWS (AWS CLI) ou le kit AWS SDK

Console de gestion AWS

La console de gestion AWS est une interface web qui permet de gérer les services AWS, offrant un accès rapide aux services, des fonctionnalités de recherche et des flux de travail simplifiés. Vous pouvez utiliser l'application mobile pour surveiller les ressources, visualiser les alarmes et vérifier la facturation, en prenant en charge plusieurs identités connectées à la fois.

Idéale pour : les utilisateurs qui préfèrent une interface visuelle et facile à utiliser pour gérer et configurer les services AWS

AWS CLI

L'AWS CLI vous permet de gérer plusieurs services AWS directement depuis la ligne de commande sous Windows, macOS et Linux. Vous pouvez automatiser les tâches à l'aide de scripts, notamment le lancement d'instances EC2.

Idéale pour : les utilisateurs avancés et les développeurs qui ont besoin d'automatiser des tâches, de scripter des actions et de gérer efficacement les ressources AWS depuis la ligne de commande

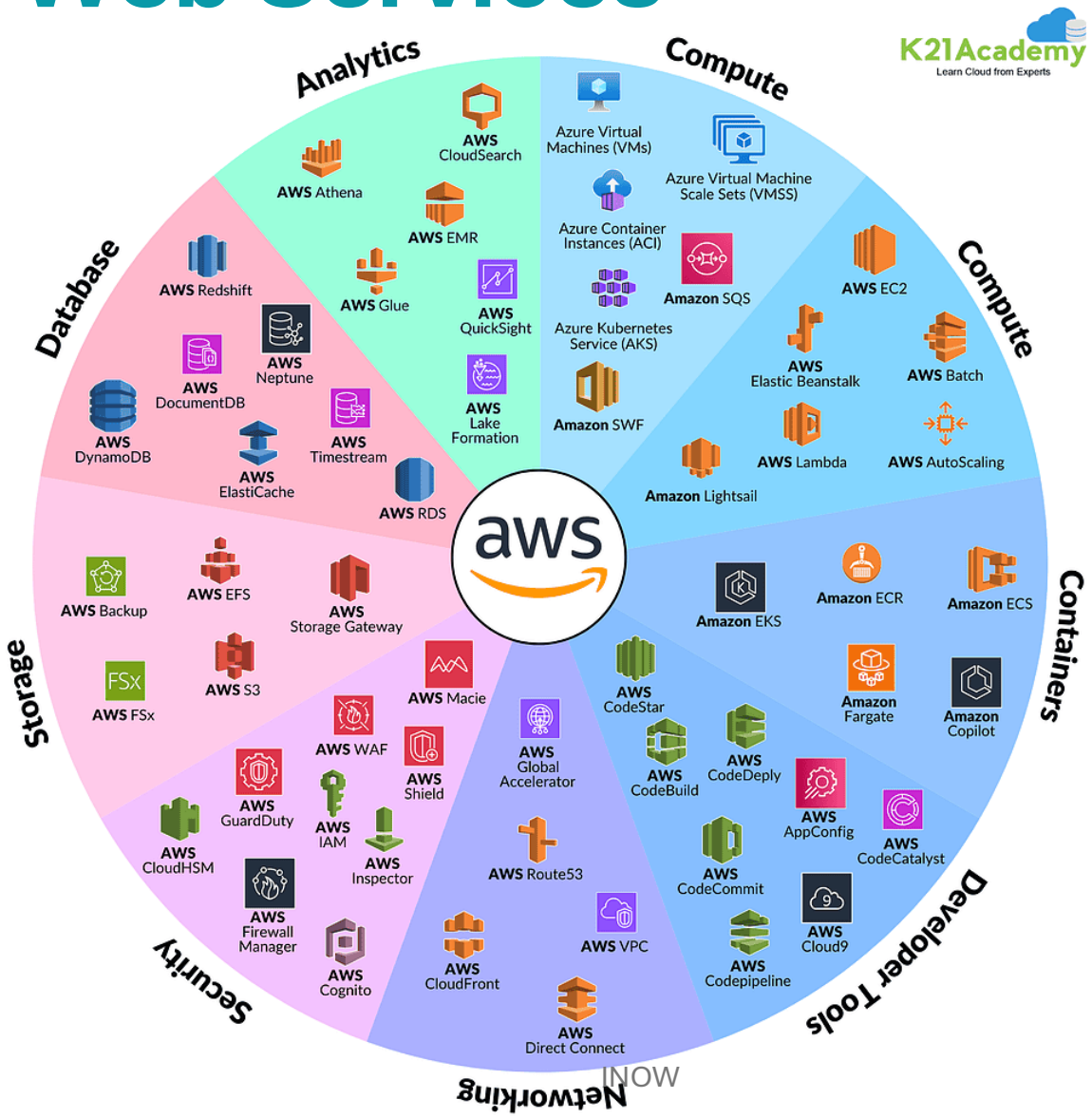
AWS SDK

Le kit AWS SDK simplifie l'intégration des services AWS dans vos applications en fournissant des API pour différents langages de programmation. AWS propose de la documentation et des exemples de code pour des langages tels que C++, Java et .NET afin de vous aider à démarrer.

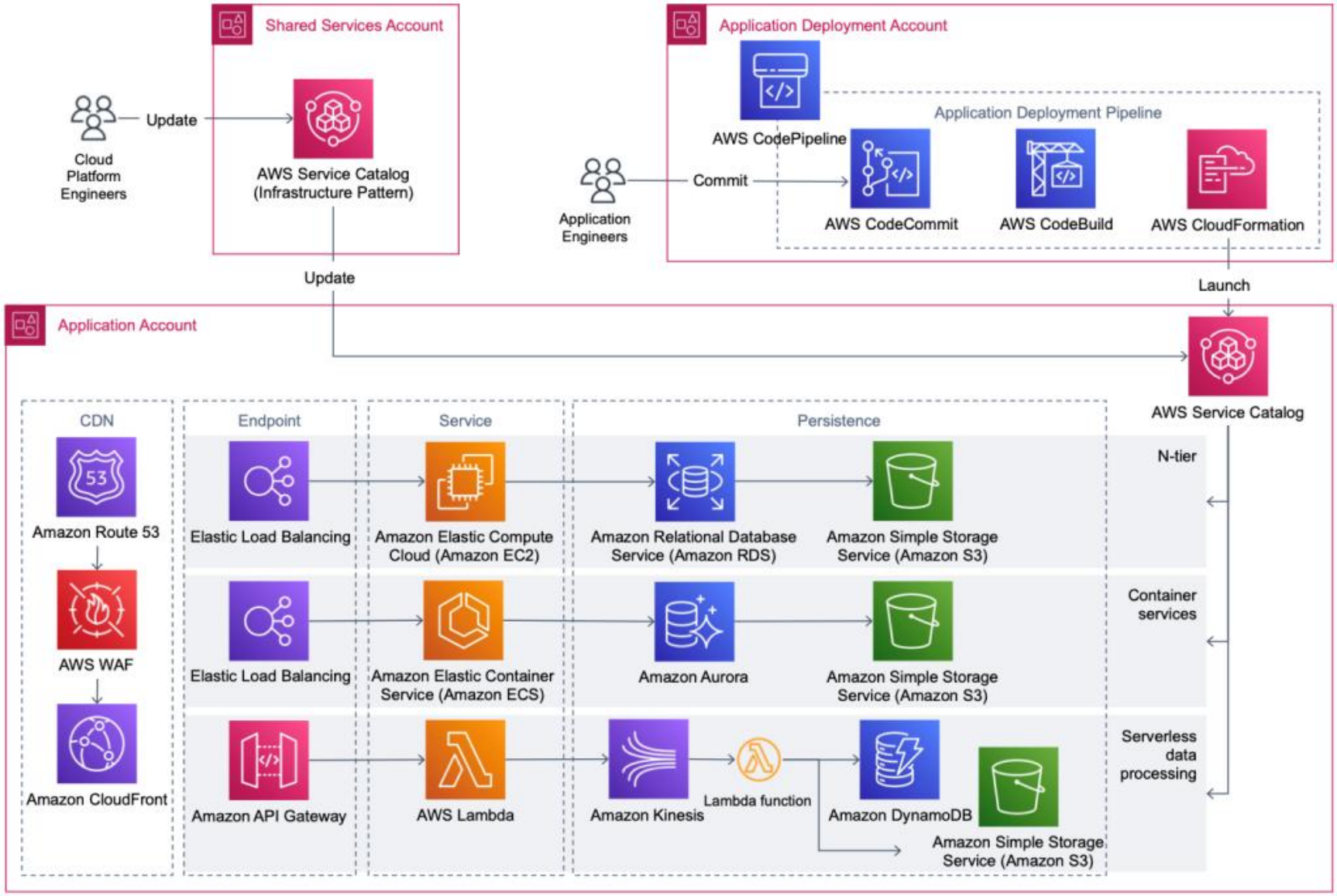
Idéale pour : les développeurs qui souhaitent intégrer les services AWS à leurs applications à l'aide d'API spécifiques à un langage

PRESENTATION D’AWS

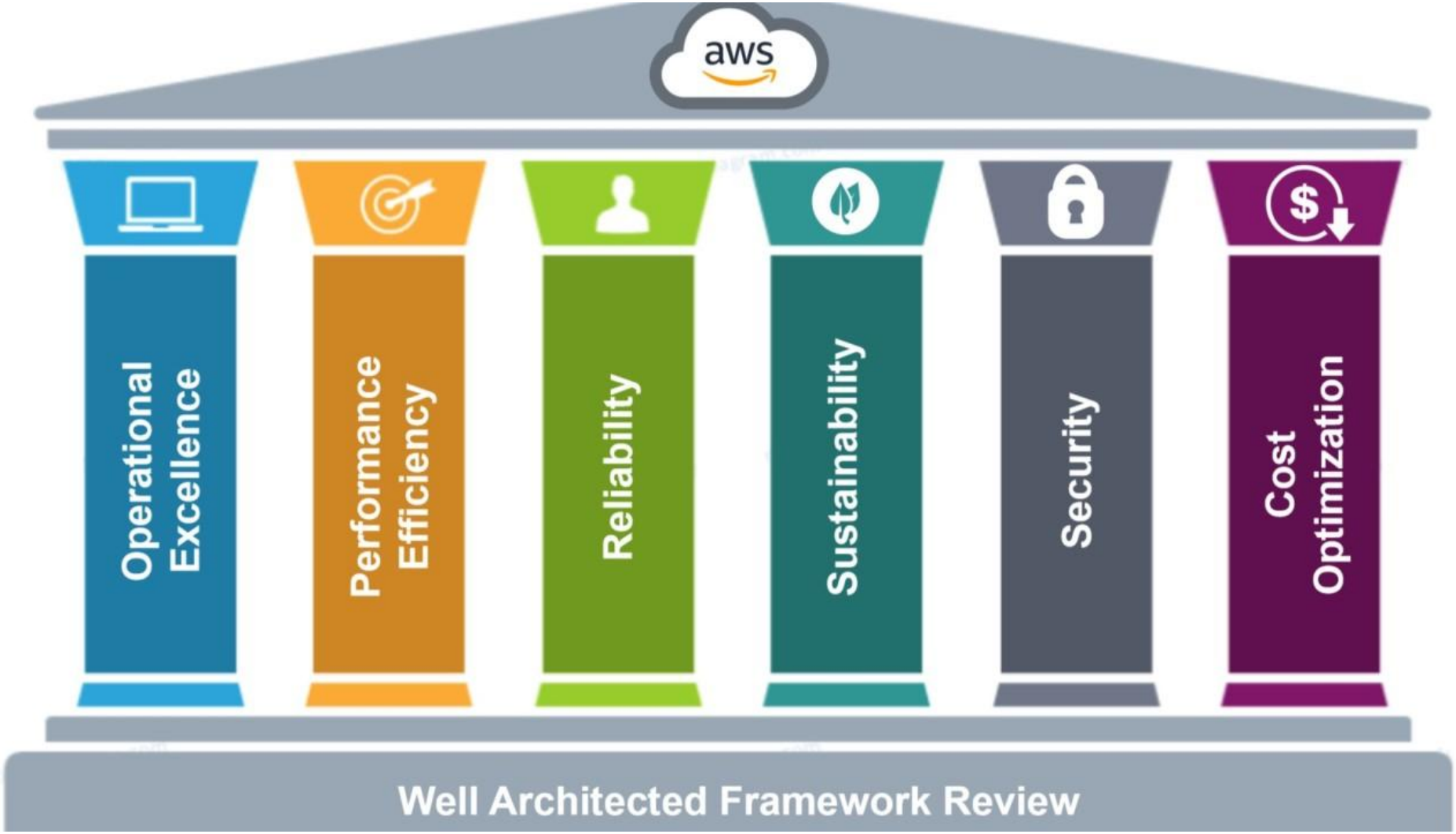
Amazon Web Services



Amazon Web Services



Amazon Web Services



CALCUL DANS LE CLOUD

AMAZON EC2

AMI

AUTO SCALING ET REPARTITION DE CHARGE

Introduction à Amazon EC2

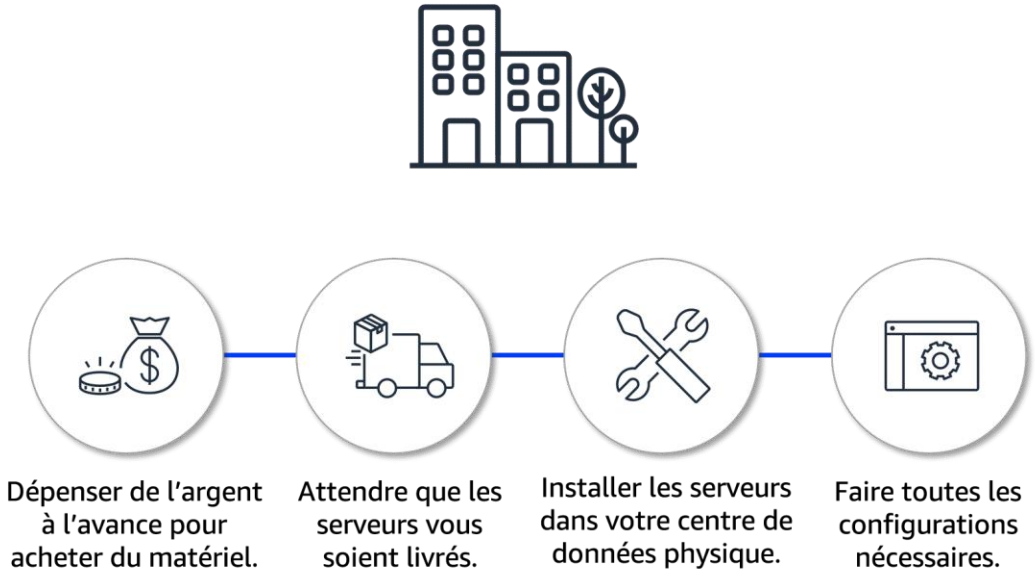
Le calcul désigne la puissance de traitement nécessaire pour exécuter des applications, gérer des données et effectuer des calculs. Dans le cloud, cette puissance est disponible à la demande. Vous pouvez y accéder à distance sans posséder ni maintenir de matériel physique. Le *calcul dans le cloud* nécessite principalement la création de machines virtuelles avec un fournisseur de cloud pour exécuter des applications et des tâches sur Internet. Dans les leçons suivantes, vous allez acquérir une compréhension approfondie d'Amazon Elastic Compute Cloud (Amazon EC2), un service de calcul puissant d'AWS, et vous découvrirez sa flexibilité, sa rentabilité et sa capacité de mise à l'échelle

Amazon EC2

- Amazon EC2 offre plus de flexibilité, de rentabilité et de rapidité que la gestion de serveurs sur site. Il offre une capacité de calcul à la demande qui peut être rapidement lancée, mise à l'échelle et résiliée, avec des coûts basés uniquement sur l'utilisation active.
- La flexibilité d'Amazon EC2 permet d'accélérer le développement et le déploiement des applications. Vous pouvez lancer aussi bien un seul serveur virtuel que plusieurs, selon vos besoins, et configurer la sécurité, la mise en réseau et le stockage. Vous pouvez également augmenter ou diminuer verticalement les ressources en fonction de leur utilisation, notamment pour gérer un trafic élevé ou des tâches à forte intensité en calcul.

Points clés à retenir : comparaison des ressources sur site et dans le cloud

Imaginez que vous êtes responsable de la conception de l'infrastructure de votre entreprise de manière à prendre en charge les nouveaux sites web. Avec les ressources sur site traditionnelles, vous devez acheter du matériel à l'avance, attendre la livraison et gérer l'installation et la configuration. Ce processus est long, coûteux et rigide, car vous êtes bloqué dans une capacité spécifique qui pourrait ne pas correspondre à l'évolution des demandes.



Avantages de l'utilisation des ressources cloud

D'autre part, Amazon EC2 vous permet de lancer, de mettre à l'échelle et d'arrêter rapidement des instances en fonction de vos besoins, sans les délais et les coûts initiaux associés aux ressources sur site traditionnelles.

Types d'instances EC2

A usage général

Les instances à usage général fournissent une combinaison équilibrée de ressources de calcul, de mémoire et de mise en réseau.

Elles sont idéales pour des charges de travail variées, notamment les services web, les référentiels de code et les cas où les performances de la charge de travail sont incertaines.

Optimisée pour le calcul

Les instances optimisées pour le calcul sont idéales pour les tâches à forte intensité de calcul, notamment les serveurs de jeu, le calcul haute performance (HPC), le machine learning et la modélisation scientifique.

Types d'instances EC2

À mémoire optimisée

Les instances à mémoire optimisée sont utilisées pour des tâches gourmandes en mémoire, notamment le traitement de grands jeux de données, l'analytique des données et les bases de données.

Elles offrent des performances rapides pour les charges de travail gourmandes en mémoire.

Calcul accéléré

Les instances de calcul accéléré utilisent des accélérateurs matériels, tels que les unités de traitement graphique (GPU), pour gérer efficacement des tâches comme les calculs à virgule flottante, le traitement de graphiques et le machine learning.

Types d'instances EC2

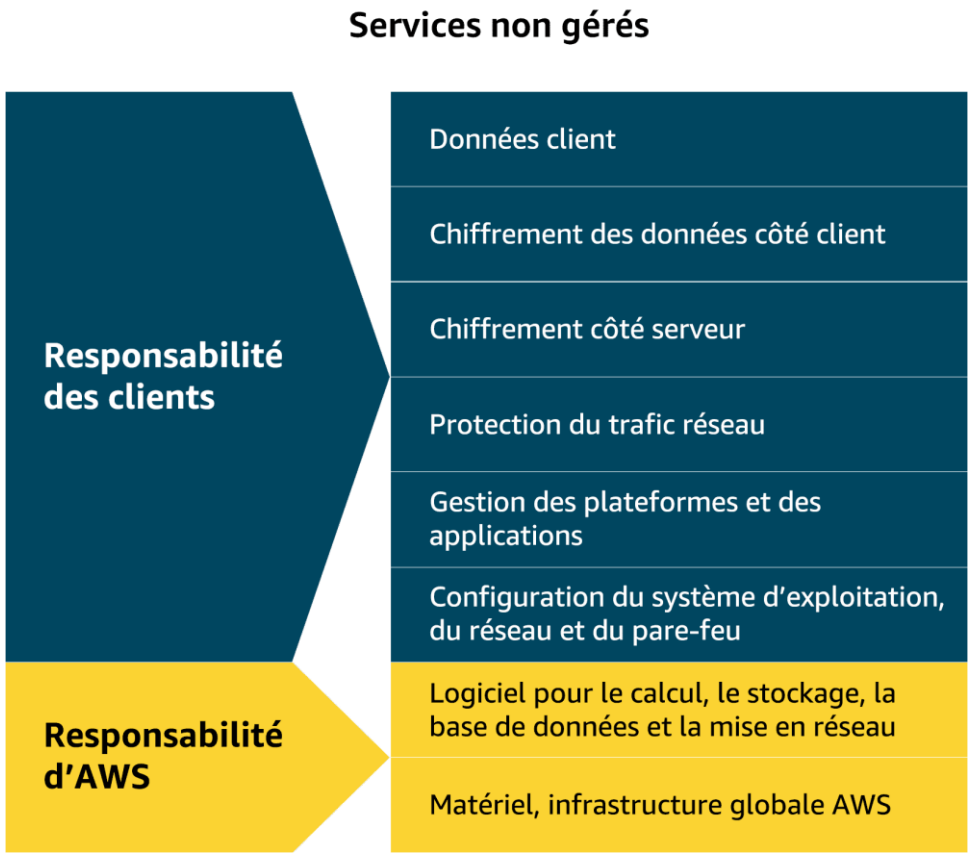
Optimisée pour le stockage

Les instances optimisées pour le stockage sont conçues pour les charges de travail nécessitant des performances élevées pour les données stockées localement, notamment les bases de données volumineuses, les entrepôts de données et les applications gourmandes en E/S.

Calcul et responsabilité partagée

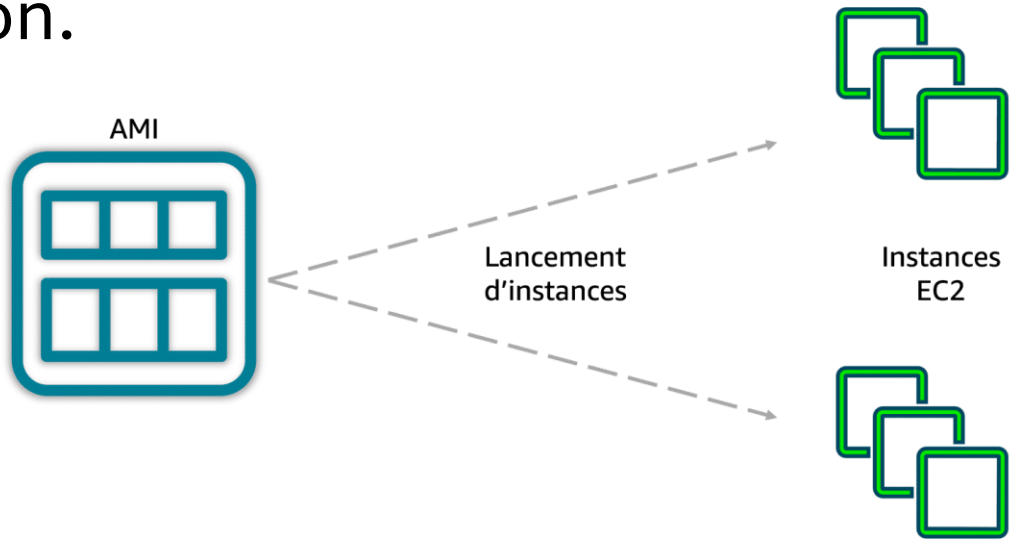
Le modèle de responsabilité partagée AWS décrit la répartition des tâches entre le client et AWS. AWS gère la sécurité *du* cloud (matériel et infrastructure), tandis que le client est responsable de la sécurité *dans* le cloud (applications, données et contrôle d'accès).

Un service *non géré* tel qu'Amazon EC2 vous oblige à effectuer toutes les tâches de configuration et de gestion de sécurité nécessaires. Lorsque vous déployez une instance EC2, vous assumez la responsabilité de configurer la sécurité, de gérer le système d'exploitation client, d'appliquer les mises à jour et de configurer les pare-feu (groupes de sécurité).



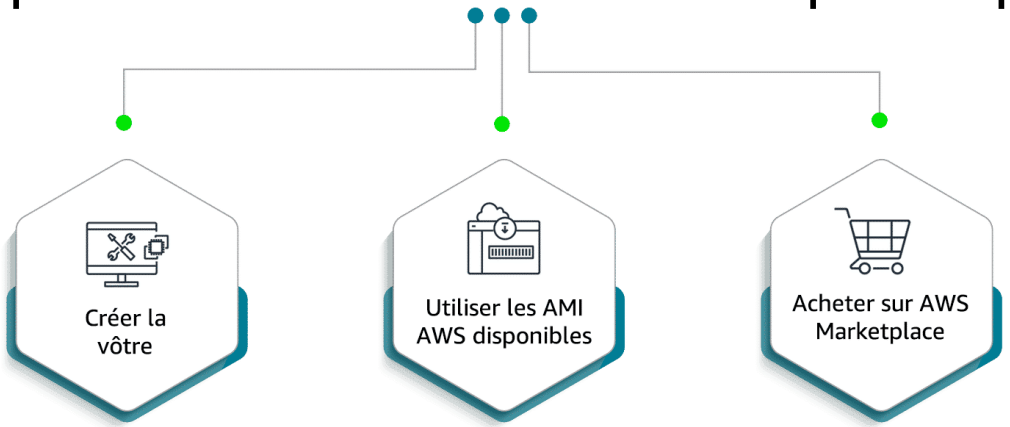
Amazon Machine Images

Une AMI comprend le système d'exploitation, la configuration du stockage, le type d'architecture, les autorisations de lancement et tout logiciel supplémentaire déjà installé. Vous pouvez utiliser une seule AMI pour lancer plusieurs instances EC2 qui ont toutes la même configuration.



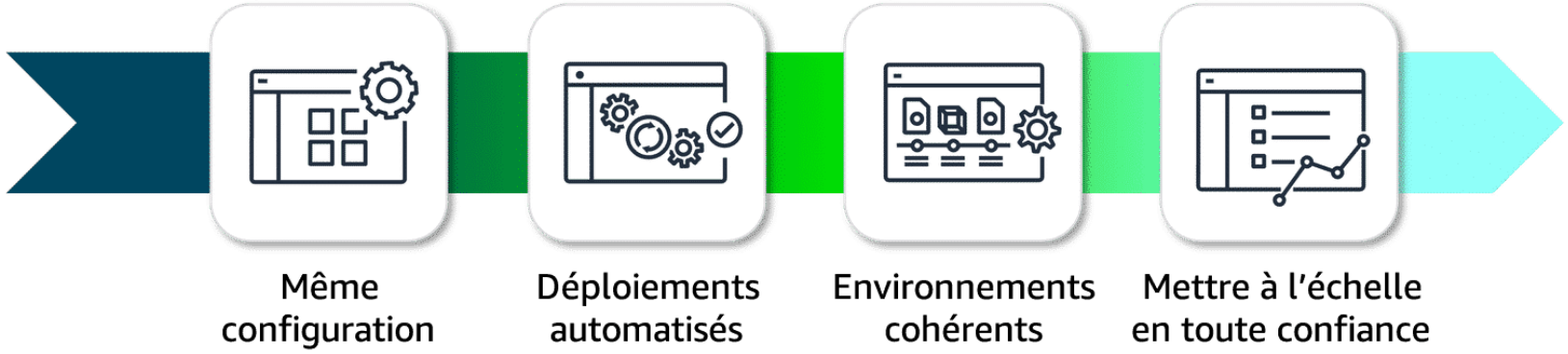
Trois moyens d'utiliser les AMI

Les AMI peuvent être utilisées de trois manières. Tout d'abord, vous pouvez créer votre propre AMI en la personnalisant avec des configurations et des logiciels spécifiques adaptés à vos besoins. Ensuite, vous pouvez utiliser des AMI AWS préconfigurées, qui sont mises en place pour des systèmes d'exploitation et des logiciels courants. Enfin, vous pouvez acheter des AMI sur AWS Marketplace, où des fournisseurs tiers proposent des logiciels spécialisés conçus pour des cas d'utilisation spécifiques



Répétabilité des AMI

Les AMI assurent la répétabilité grâce à un environnement cohérent pour chaque nouvelle instance. Comme les configurations sont identiques et les déploiements automatisés, les environnements de développement et de test sont cohérents. Cela facilite la mise à l'échelle, réduit les erreurs et simplifie la gestion des environnements à grande échelle.



Options de tarification d'AWS

Instance à la demande

Payez uniquement pour la capacité de calcul que vous consommez, sans paiement à l'avance ni engagement à long terme.

Instances réservées

Réalisez des économies allant jusqu'à 75 % en vous engageant sur une durée de 1 an ou 3 ans pour des charges de travail prévisibles en utilisant des familles d'instances et des régions AWS spécifiques.

Instances Spot

Enchérissez sur la capacité de calcul de réserve en bénéficiant d'une réduction allant jusqu'à 90 % sur le prix à la demande, avec la flexibilité de pouvoir interrompre le service lorsque AWS récupère l'instance.

Options de tarification d'AWS

Savings Plans

Bénéficiez d'économies allant jusqu'à 72 % sur une variété de types d'instances et de services en vous engageant à maintenir un niveau d'utilisation cohérent pendant 1 an ou 3 ans.

Hôtes dédiés

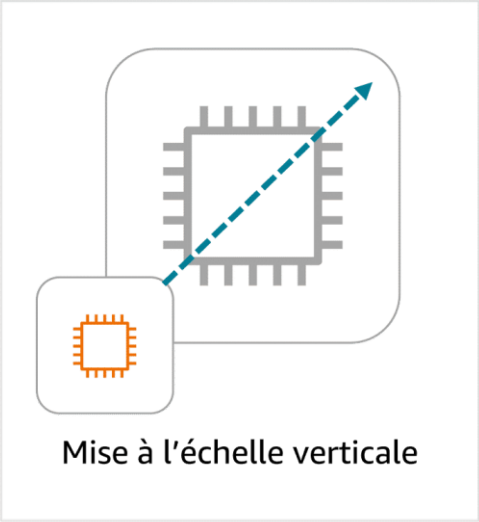
Réservez un serveur physique complet exclusivement pour votre usage. Cette option permet d'avoir un contrôle total et est idéale pour les charges de travail soumises à des exigences strictes en matière de sécurité ou de licence.

Instances dédiées

Payez pour les instances exécutées sur un matériel dédié uniquement à votre compte. Cette option favorise l'isolement des autres clients AWS.

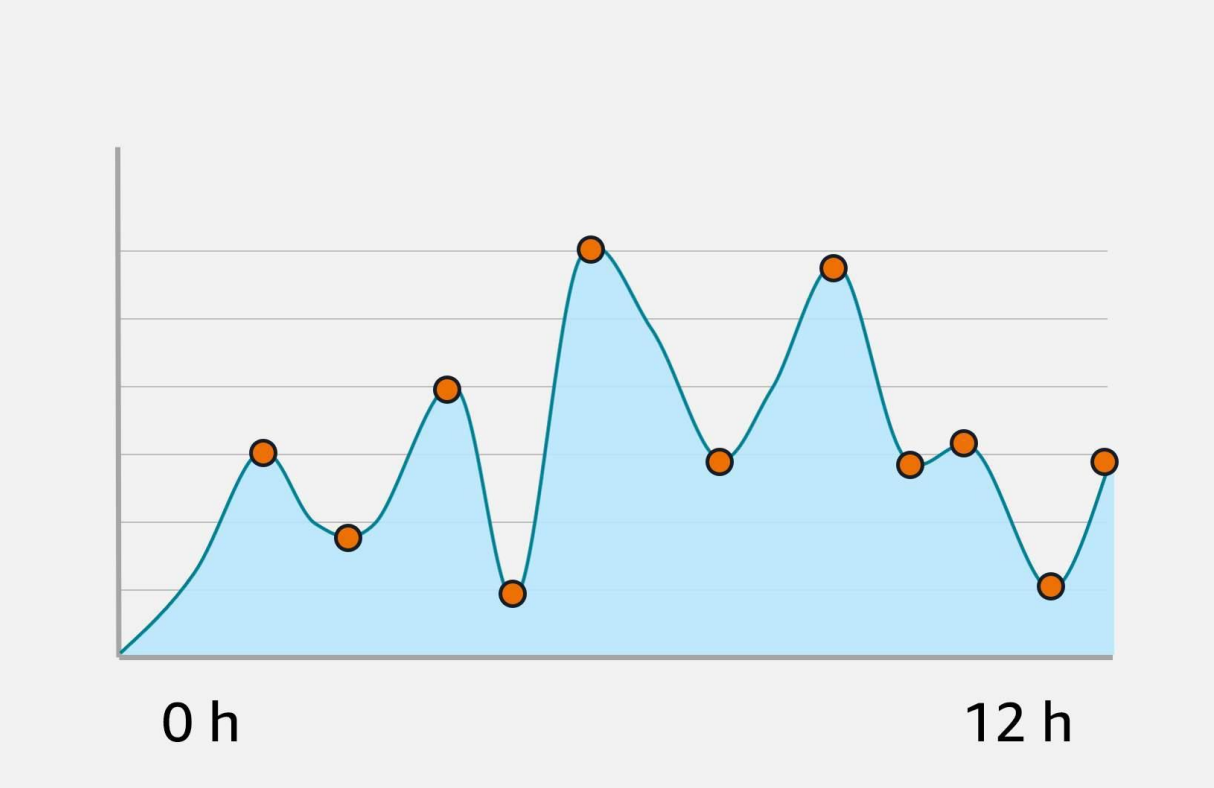
Mise à l'échelle d'Amazon EC2

La capacité de mise à l'échelle désigne la capacité d'un système à gérer une charge accrue en ajoutant des ressources. Vous pouvez augmenter un système verticalement en ajoutant de la puissance aux machines existantes, ou l'augmenter horizontalement en ajoutant d'autres machines. La capacité de mise à l'échelle est axée sur la planification des capacités à long terme afin de s'assurer que le système peut évoluer et accueillir davantage d'utilisateurs ou de charges de travail, selon les besoins.



Élasticité

L'élasticité désigne la capacité à agrandir ou réduire automatiquement les ressources en fonction de la demande en temps réel. Un système peut donc ajuster rapidement ses ressources, avec une mise à l'échelle horizontale pendant les périodes de forte demande et une mise à l'échelle de réduction lorsque la demande diminue. L'élasticité assure une rentabilité et une utilisation optimale des ressources à tout moment.



Amazon EC2 Auto Scaling

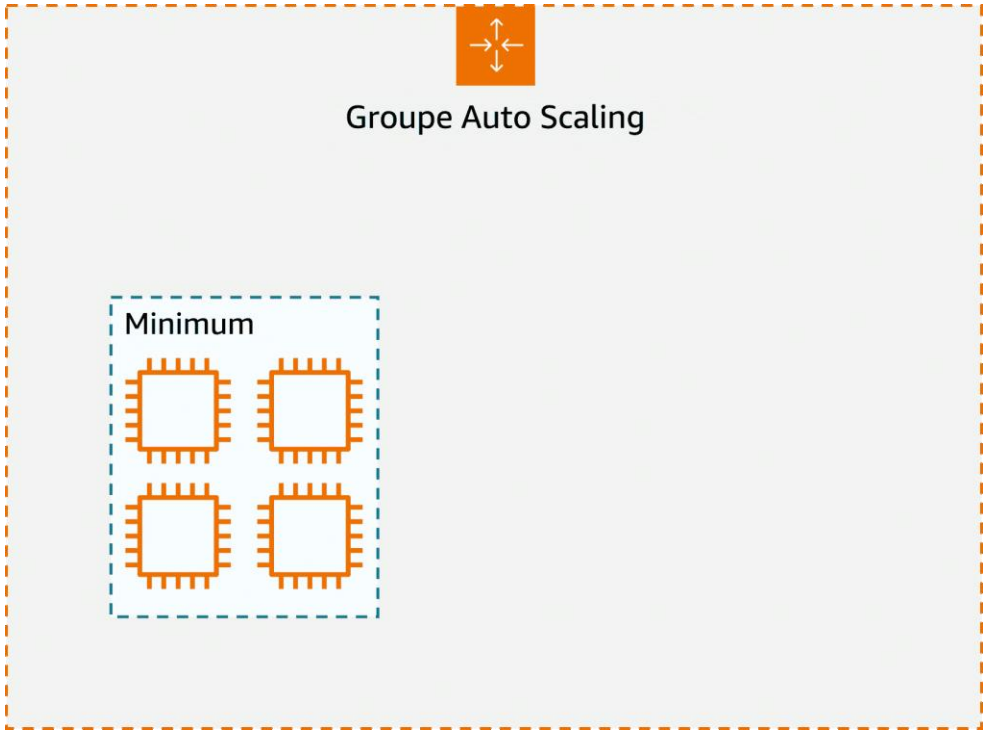
EC2 Auto Scaling vous permet de maintenir la capacité de calcul souhaitée pour votre application en ajustant dynamiquement le nombre d'instances EC2 en fonction de la demande. Vous pouvez créer des groupes Auto Scaling, qui sont des ensembles d'instances EC2 capable de se réduire ou d'augmenter horizontalement pour répondre aux besoins de votre application.

Un groupe Auto Scaling est configuré avec les trois paramètres clés suivants.

Amazon EC2 Auto Scaling
Capacité minimale

La *capacité minimale* définit le nombre minimal d'instances EC2 nécessaires pour que l'application continue de fonctionner. Cela garantit que le système ne se mette jamais à l'échelle en dessous de ce seuil. Il s'agit du nombre d'instances EC2 lancées immédiatement après la création du groupe Auto Scaling.

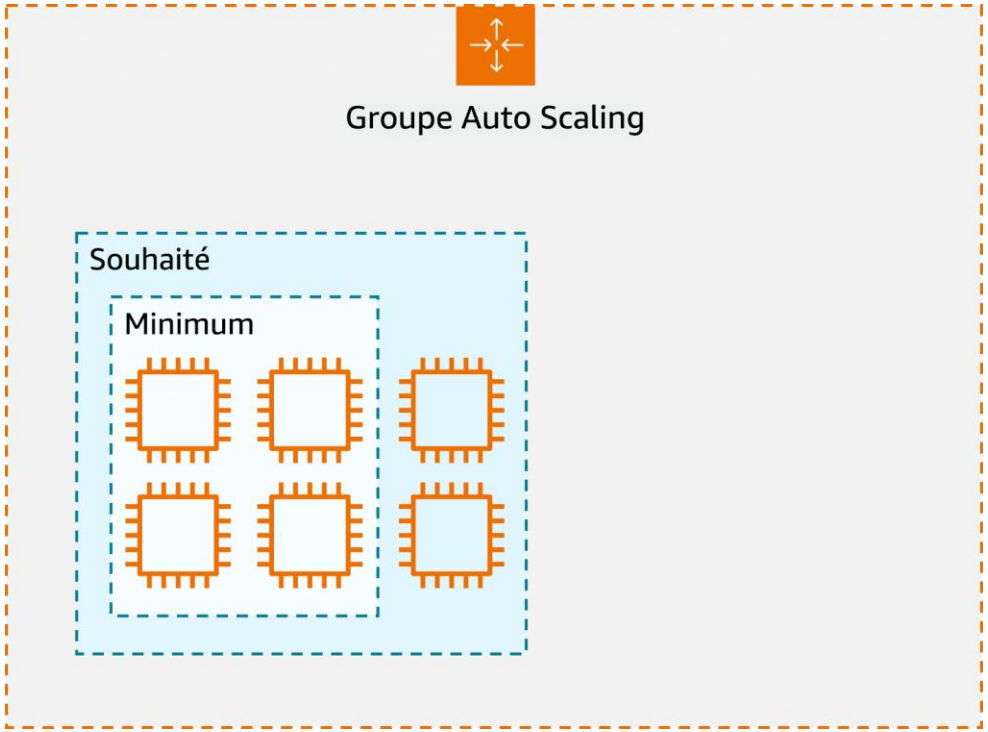
Dans cet exemple, la capacité minimale est de quatre instances EC2.



Amazon EC2 Auto Scaling Capacité souhaitée

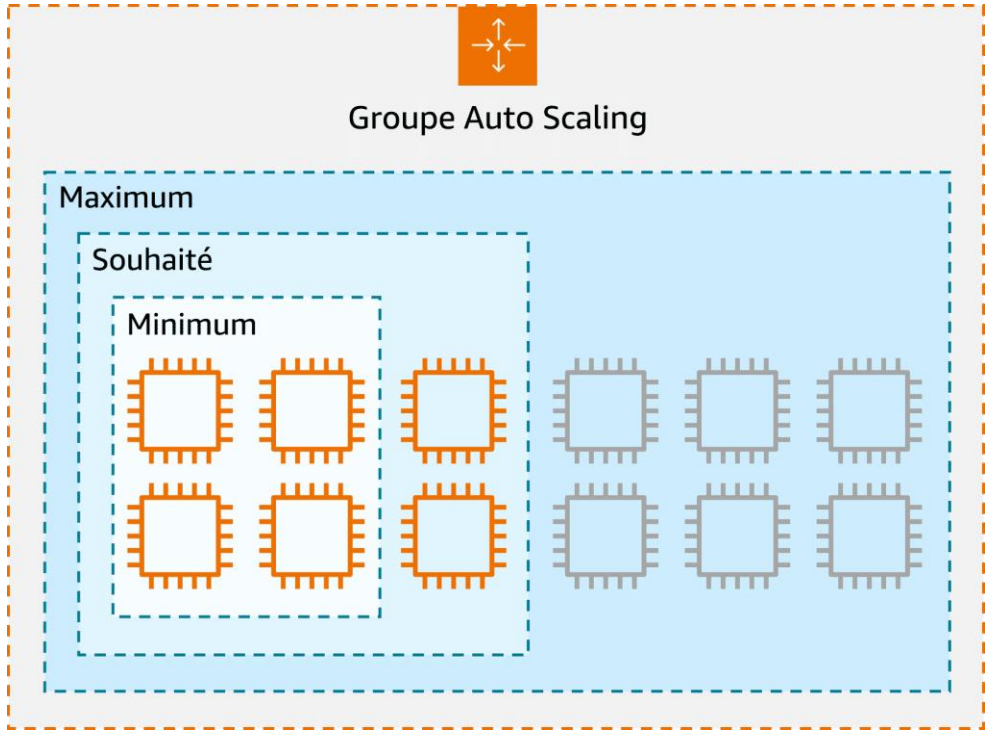
La *capacité souhaitée* est le nombre idéal d'instances nécessaires pour gérer la charge de travail actuelle, qu'Auto Scaling prévoit maintenir. Si vous ne spécifiez pas le nombre souhaité d'instances EC2 dans un groupe Auto Scaling, la capacité souhaitée sera égale par défaut à votre capacité minimale.

Dans cet exemple, la capacité souhaitée est de six instances EC2.



Amazon EC2 Auto Scaling
Capacité maximale

La *capacité maximale* fixe une limite supérieure au nombre d'instances pouvant être lancées, afin d'éviter une mise à l'échelle excessive et de contrôler les coûts. Par exemple, vous pouvez configurer le groupe Auto Scaling pour augmenter horizontalement pour répondre à une demande accrue.



Dans cet exemple, il est possible de lancer un maximum de 12 instances EC2. Amazon EC2 Auto Scaling se mettra à l'échelle entre le nombre minimum et le nombre maximum d'instances.

Elastic Load Balancing

Elastic Load Balancing (ELB) distribue automatiquement le trafic d'application entrant sur plusieurs ressources, telles que les instances EC2, afin d'optimiser les performances et la fiabilité. Un équilibreur de charge sert de point de contact unique pour tout le trafic web entrant vers un groupe Auto Scaling. Comme le nombre d'instances EC2 varie en fonction des demandes de trafic, les demandes entrantes sont d'abord dirigées vers l'équilibreur de charge. De là, le trafic est distribué uniformément entre les instances disponibles.

Bien qu'ELB et Amazon EC2 Auto Scaling soient des services distincts, ils fonctionnent en tandem pour améliorer les performances des applications et garantir une haute disponibilité. Ensemble, ils permettent aux applications exécutées sur Amazon EC2 de se mettre à l'échelle efficacement tout en maintenant des performances cohérentes.

Avantages de l’ELB

Distribution efficace du trafic

ELBdistribue uniformément le trafic entre les instances EC2, évitant ainsi de surcharger une seule instance et optimisant l'utilisation des ressources.

Mise à l'échelle automatique

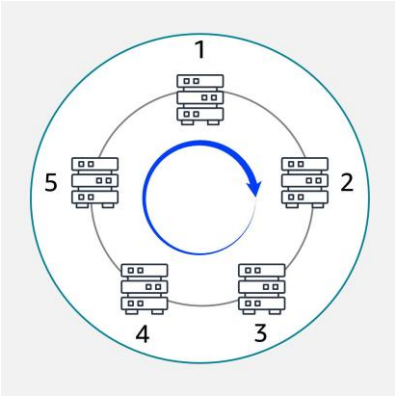
ELB semet à l'échelle en fonction du trafic et s'ajuste automatiquement à l'évolution de la demande pour un fonctionnement harmonieux au fur et à mesure que les instances backend sont ajoutées ou supprimées.

Gestion simplifiée

ELB découpleles niveaux frontaux et backend et réduit la synchronisation manuelle. Il gère également la maintenance, les mises à jour et le basculement afin de réduire les frais opérationnels.

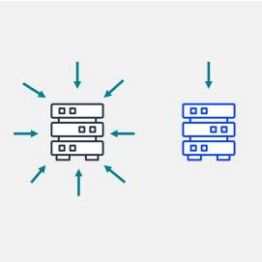
Méthodes de routage

Pour optimiser la distribution du trafic, ELB recourt à plusieurs méthodes de routage : tourniquet, moins de connexions, hachage IP, temps de réponse le plus court. Ces stratégies de routage fonctionnent ensemble pour assurer une gestion efficace du trafic et des performances optimales des applications.



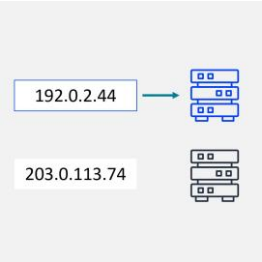
Tourniquet
Répartit le trafic uniformément sur tous les serveurs disponibles de manière cyclique.

Méthodes de routage



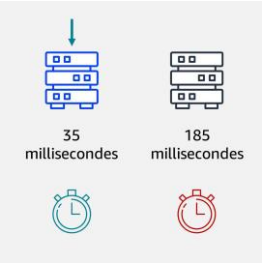
Moins de connexions

Achemine le trafic vers le serveur comportant le moins de connexions actives, en maintenant une charge équilibrée.



Hachage IP

Utilise l'adresse IP du client pour acheminer le trafic de manière cohérente vers le même serveur.



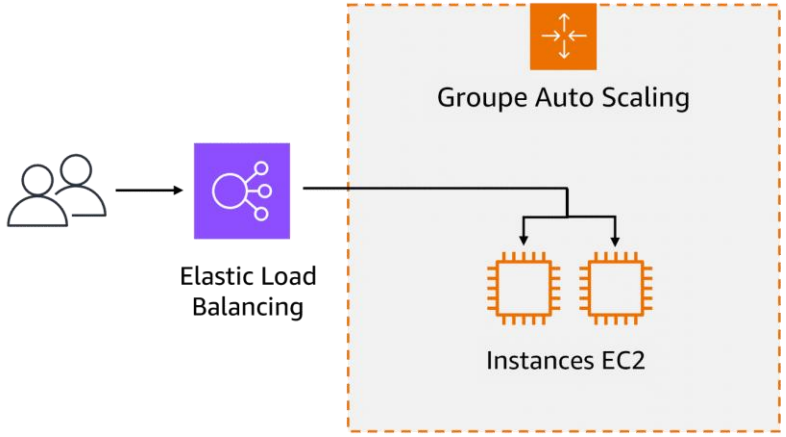
Temps de réponse le plus court

Dirige le trafic vers le serveur ayant le temps de réponse le plus rapide, ce qui permet de minimiser la latence.

ELB : Configuration initiale

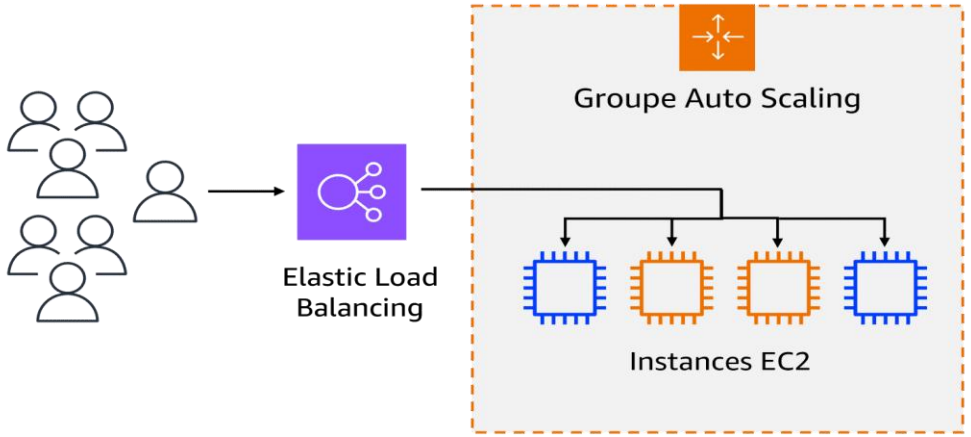
Prenons un exemple pour mieux comprendre le fonctionnement d'Elastic Load Balancing dans le cloud computing. Dans le secteur de la santé, en particulier dans les hôpitaux et les établissements médicaux qui disposent de systèmes de prise de rendez-vous en ligne ou des portails pour les patients, le trafic du site web peut varier considérablement au cours de la journée.

Période de faible demande : au début de la journée, seuls quelques patients accèdent au système pour prendre rendez-vous ou consulter leurs dossiers médicaux. Les serveurs web existants sont suffisants pour gérer le faible trafic. Cela correspond à la demande, sans nécessiter de ressources supplémentaires à ce stade.



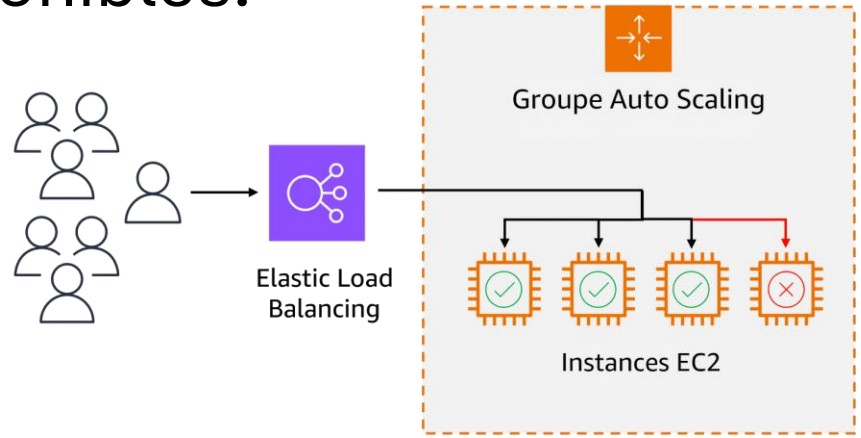
ELB: Mise à l’échelle verticale

Période de forte demande : à mesure que la journée avance, en particulier aux heures de pointe, comme tôt le matin ou juste avant le week-end, de plus en plus de patients accèdent au portail pour prendre rendez-vous, consulter les résultats des tests ou contacter des professionnels de santé. Pour faire face à cette augmentation soudaine de la demande, le système de santé augmente verticalement et automatiquement le nombre de serveurs afin de garantir la réactivité et la disponibilité du système pour tous les utilisateurs.



ELB: Répartition de charge

Un équilibreur de charge oriente le trafic entrant vers différents serveurs web en fonction de leur charge actuelle. Par exemple, si un serveur commence à recevoir trop de demandes, l'équilibreur de charge achemine les nouvelles demandes vers un serveur moins occupé. Cette méthode permet de veiller à ce qu'aucun serveur ne soit submergé. Le trafic est distribué uniformément entre les instances EC2 disponibles.



TP 1: Architecture scalable avec AWS EC2, Auto Scaling Group et Elastic Load Balancer

Contexte

Vous êtes **ingénieur cloud** pour une entreprise fictive nommée **KaruKlass**.

L'entreprise souhaite déployer un **site web public** capable de :

- supporter une montée en charge soudaine,
- rester disponible en cas de panne d'instance,
- s'adapter automatiquement à la demande,
- être supervisé via les outils AWS.

Votre mission est de **concevoir et déployer une architecture scalable et hautement disponible sur AWS**, en utilisant uniquement des **services managés standards**.

TP 1: Architecture scalable avec AWS EC2, Auto Scaling Group et Elastic Load Balancer

Architecture cible (description fonctionnelle)

- L'architecture attendue doit respecter les principes suivants :
- Un **Load Balancer public** reçoit tout le trafic HTTP
- Plusieurs **instances EC2** hébergent le site web
- Les instances sont gérées automatiquement par un **Auto Scaling Group**
- Le nombre d'instances varie selon la charge
- Toute instance défaillante est automatiquement remplacée

L'architecture doit être **déployée dans au moins deux zones de disponibilité**.

TP 1: Architecture scalable avec AWS EC2, Auto Scaling Group et Elastic Load Balancer

Contraintes techniques :

- Région AWS : **eu-west-X**
- Utiliser une **AMI Amazon Linux récente**
- Type d'instance compatible **Free Tier**
- Accès SSH **restreint** (pas ouvert à tout Internet)
- Le site web doit afficher :
  - un message explicite
  - l'identifiant de l'instance (pour observer la répartition de charge)

TP 1: Architecture scalable avec AWS EC2, Auto Scaling Group et Elastic Load Balancer

À la fin du TP, vous devez être en mesure de fournir :

- L'URL publique du Load Balancer
- Une page web accessible depuis un navigateur
- Une preuve de répartition de charge (changement d'instance affichée)
- Une démonstration de mise à l'échelle automatique
- Une observation des métriques dans CloudWatch

Scénarios de validation

Vous devez être capable de démontrer que :

- Le site reste accessible si une instance EC2 est supprimée manuellement
- Le nombre d'instances augmente sous charge
- Le Load Balancer distribue bien les requêtes
- Les métriques CPU et trafic sont visibles dans CloudWatch

TP 1: Architecture scalable avec AWS EC2, Auto Scaling Group et Elastic Load Balancer

Questions de réflexion

Répondez aux questins suivantes :

- Pourquoi ne pas exposer directement une instance EC2 sur Internet ?
- Quelle est la différence entre **scalabilité** et **élasticité** ?
- Que se passe-t-il si une instance tombe en panne ?
- Quel composant est responsable de l'auto-healing ?
- Quels sont les impacts de cette architecture sur les **coûts** ?

Pour aller plus loin (optionnel)

Si vous avez terminé :

- Simuler une montée en charge depuis CloudShell
- Observer l'évolution du nombre d'instances
- Modifier la version de l'application et déclencher un **rolling update**
- Réfléchir à :
  - l'ajout du HTTPS
  - la centralisation des logs
  - le déploiement via Infrastructure as Code

TP 1: Architecture scalable avec AWS EC2, Auto Scaling Group et Elastic Load Balancer

À la fin du TP, vous devez être en mesure de fournir :

- L'URL publique du Load Balancer
- Une page web accessible depuis un navigateur
- Une preuve de répartition de charge (changement d'instance affichée)
- Une démonstration de mise à l'échelle automatique
- Une observation des métriques dans CloudWatch

Messagerie et mise en file d'attente

Messagerie et mise en file d'attente

Amazon EventBridge, Amazon SNS et Amazon SQS sont des services AWS qui permettent aux différentes parties d'une application de communiquer efficacement dans le cloud.

Ces services permettent de créer des systèmes pilotés par des événements et basés sur des messages.

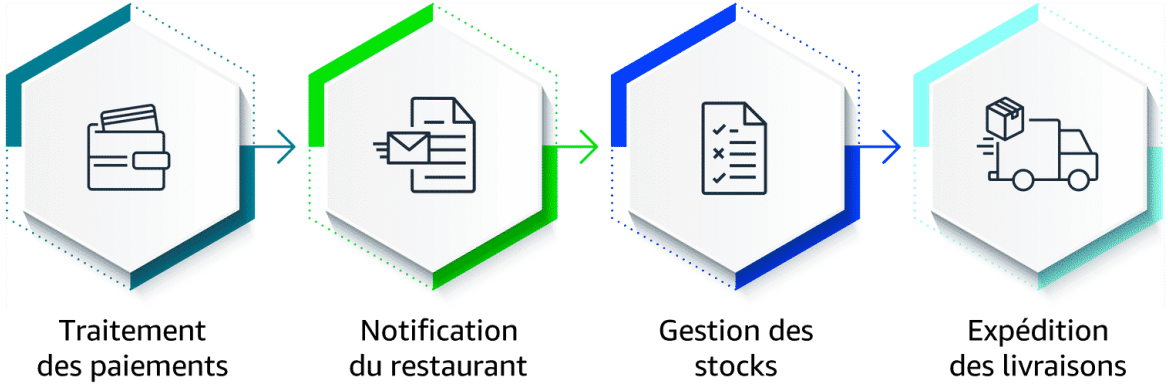
Ensemble, ils contribuent à créer des applications fiables pouvant être mises à l'échelle et capables de gérer un trafic élevé et d'améliorer la communication entre les composants.

EventBridge

EventBridge est un service sans serveur qui permet de connecter les différentes parties d'une application à l'aide d'événements, ce qui permet de créer des systèmes pouvant être mis à l'échelle et pilotés par les événements.

Avec EventBridge, vous acheminez des événements provenant de sources telles que des applications personnalisées, des services AWS et des logiciels tiers vers d'autres applications.

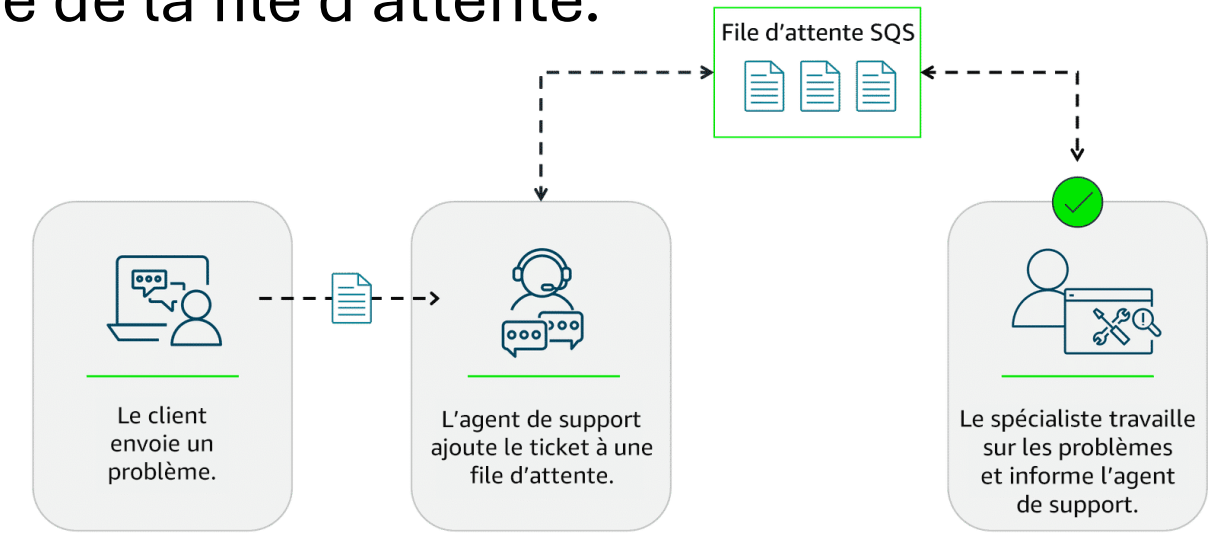
EventBridge simplifie le processus de réception, de filtrage, de transformation et de diffusion des événements, ce qui vous permet de créer rapidement des applications fiables.



Amazon SQS

Amazon SQS est un service de mise en file d'attente de messages qui facilite la communication fiable entre les composants logiciels. Il peut envoyer, stocker et recevoir des messages à n'importe quelle échelle, en veillant à ce qu'aucun message ne soit perdu et qu'aucun autre service ne soit nécessaire pour leur traitement.

Dans Amazon SQS, une application place les messages dans une file d'attente et un utilisateur ou un service récupère le message, le traite, puis le supprime de la file d'attente.



Amazon SNS

Amazon SNS est un service de publication-abonnement que les diffuseurs de publications utilisent pour envoyer des messages aux abonnés via des rubriques SNS.

Dans Amazon SNS, les abonnés peuvent inclure des serveurs web, des adresses e-mail, des fonctions Lambda et divers autres points de terminaison.

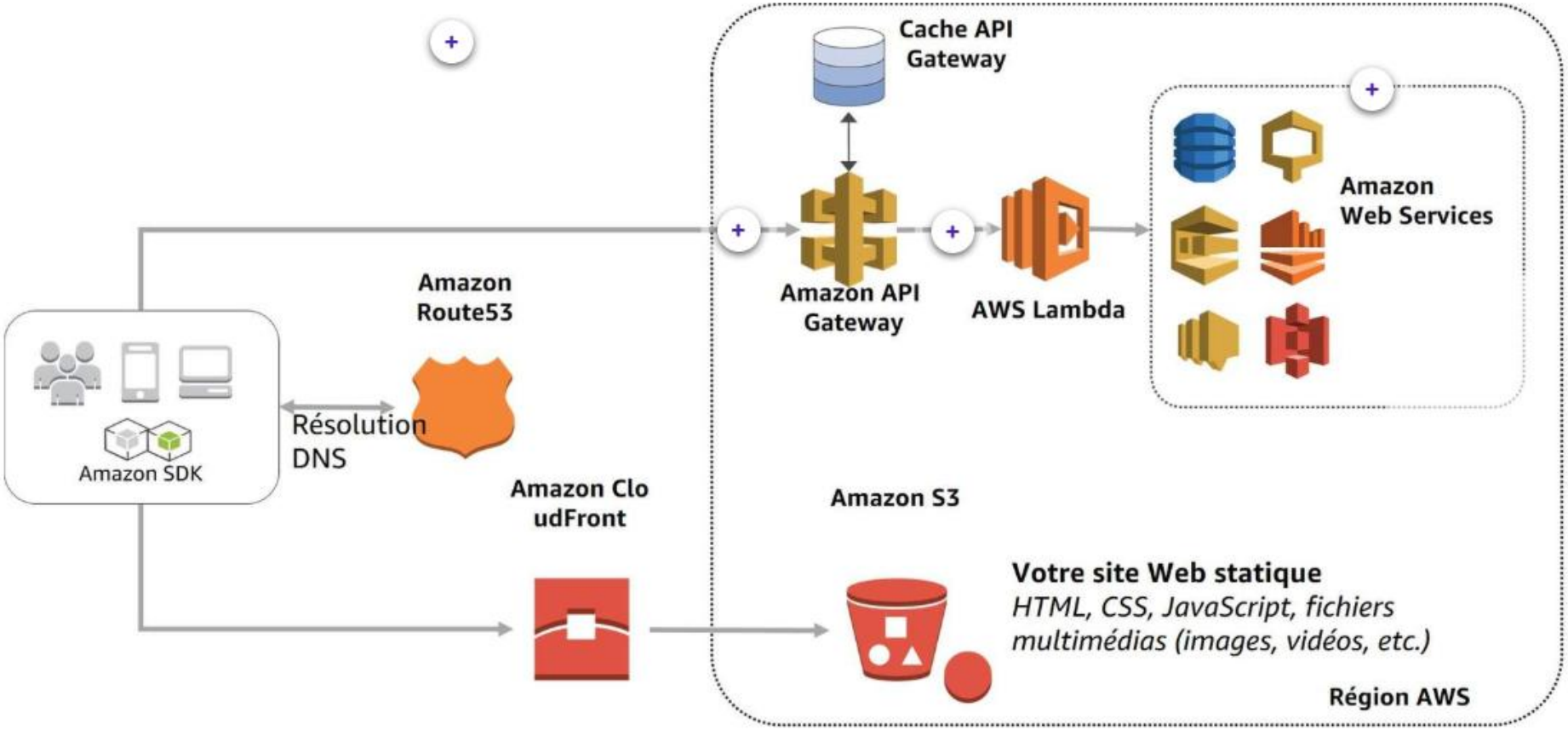
AWS LAMBDA

AWS Lambda est le service de calcul pour l'informatique serverless

La plateforme serverless d'AWS comprend également un certain nombre de services entièrement étroitement intégrés avec Lambda et parfaitement adaptés aux applications serverless.

Il y a également des outils de développement, dont le modèle SAM (Serverless Application Model), qui simplifient le déploiement de vos fonctions Lambda et de vos applications serverless.

AWS LAMBDA


Cache API Gateway
Amazon Web Services
Amazon API Gateway
AWS Lambda
Amazon Route53
Résolution DNS
Amazon SDK
Amazon CloudFront
Amazon S3
Votre site Web statique
HTML, CSS, JavaScript, fichiers multimédias (images, vidéos, etc.)
Région AWS

Caractéristiques d’AWS LAMBDA

- Le code que vous écrivez pour Lambda ne l'est pas dans un nouveau langage que vous devez apprendre. Le développement dans Lambda n'est pas étroitement lié à AWS, de sorte que vous pouvez aisément porter du code dans et hors d'AWS.
- Dans votre fonction Lambda, vous pouvez faire tout ce que les applications traditionnelles peuvent faire, notamment appeler un kit SDK AWS ou une API tierce, que ce soit sur AWS, dans votre centre de données ou sur Internet.
- Au lieu de mettre à l'échelle en ajoutant des serveurs, Lambda évolue en réactions aux événements. Vous configurez les paramètres de mémoire et AWS gère les détail,s tels que le CPU, le réseau et le débit d'E/S.
- Le modèle d'autorisations de Lambda utilise AWS Identity & Access Management (IAM) pour accorder l'accès en toute sécurité aux ressources souhaitées et vous donner un contrôle précis de l'appel de vos fonctions.
- Comme Lambda est un service entièrement géré, la haute disponibilité et la tolérance aux pannes sont intégrées au service sans aucune configuration supplémentaire de votre part.
- Les fonctions Lambda ne s'exécutant que lorsque vous les déclenchez, vous ne payez jamais pour la capacité inactive.

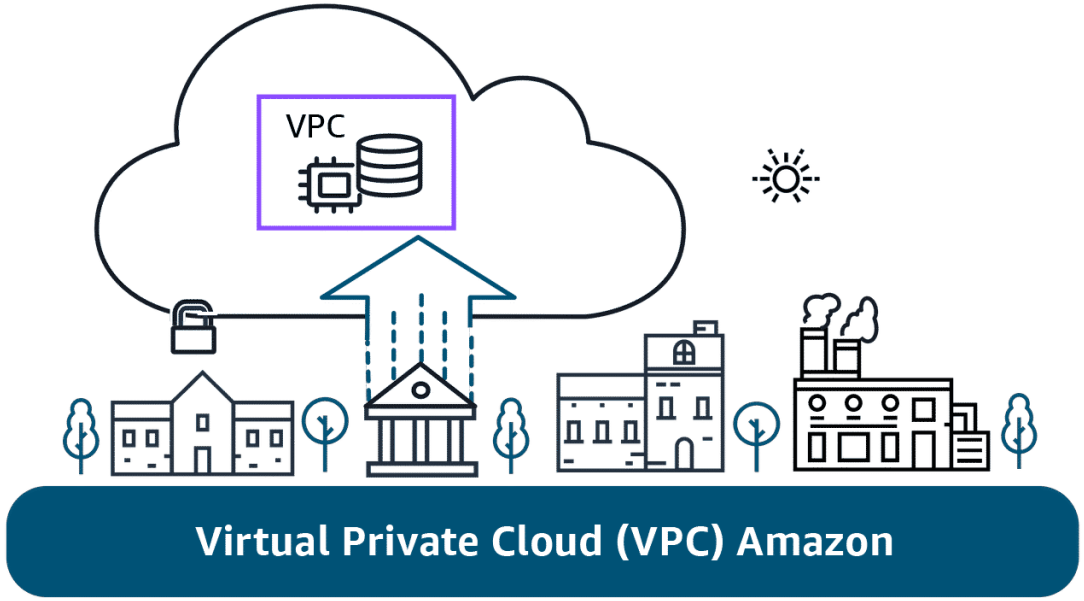
Network

Présentation de la mise en réseau

Le terme *mise en réseau* fait référence à des appareils interconnectés qui peuvent échanger des données et des ressources. La mise en réseau dans le cloud AWS comprend l'infrastructure et les services qui fonctionnent ensemble pour héberger vos applications, vos données et toutes les autres ressources dont vous pourriez avoir besoin.

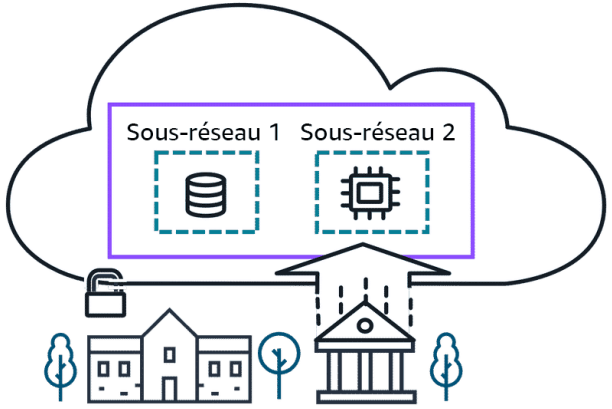
Virtual Private Cloud Amazon (Amazon VPC)

Un Amazon VPC vous permet de mettre en service une section logiquement isolée du cloud AWS où vous pouvez lancer des ressources AWS dans un réseau virtuel que vous définissez.



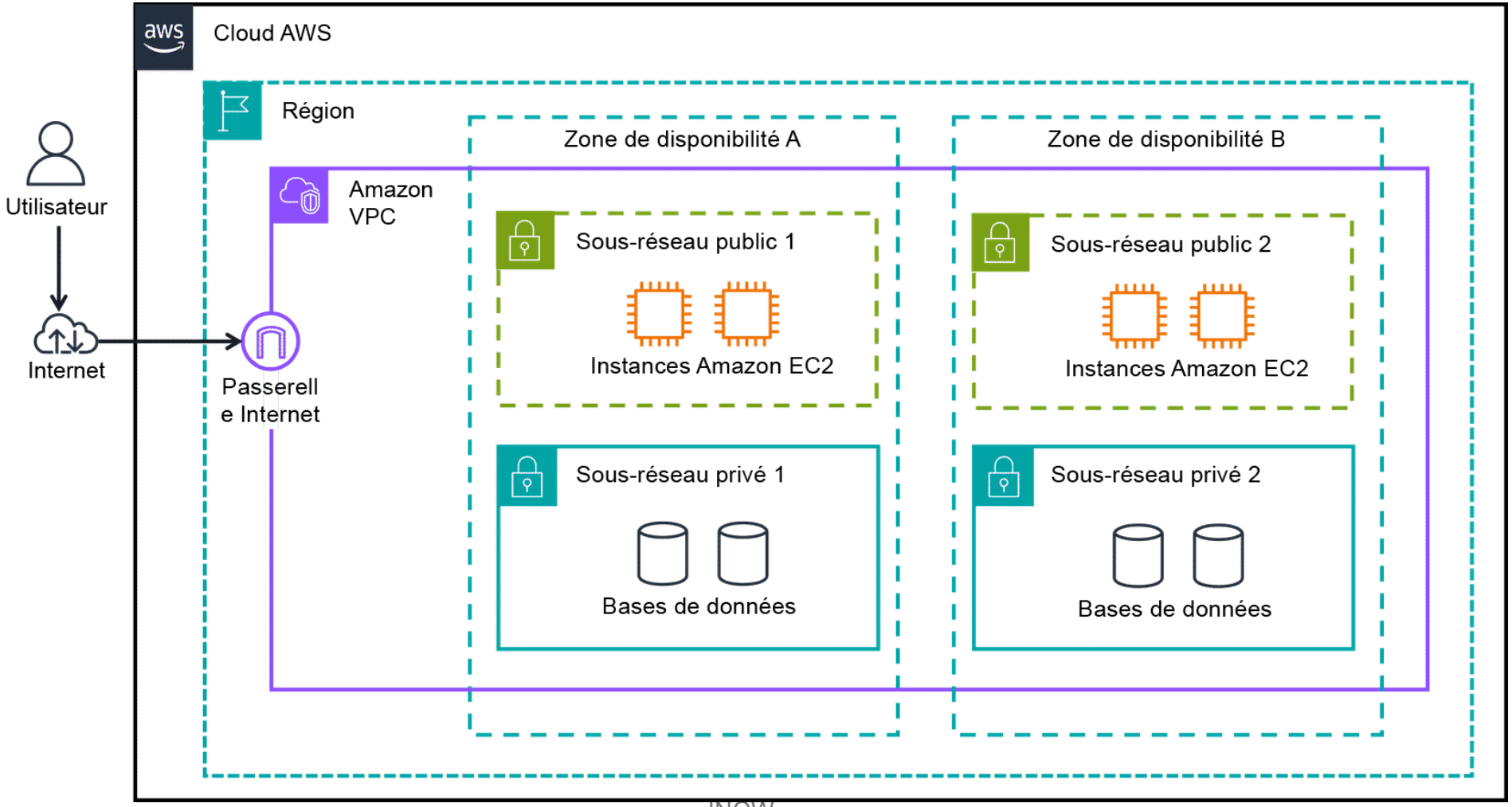
Sous-réseau

Les sous-réseaux sont utilisés pour organiser vos ressources et peuvent être rendus accessibles au public ou en privé. Un sous-réseau privé est couramment utilisé pour contenir des ressources telles qu'une base de données stockant des informations sur les clients ou les transactions. Un sous-réseau public est couramment utilisé pour des ressources telles qu'un site web destiné aux clients.



Sous-réseau : une sous-section d'un VPN

Composants réseau : comprendre les connexions à l'aide de diagrammes



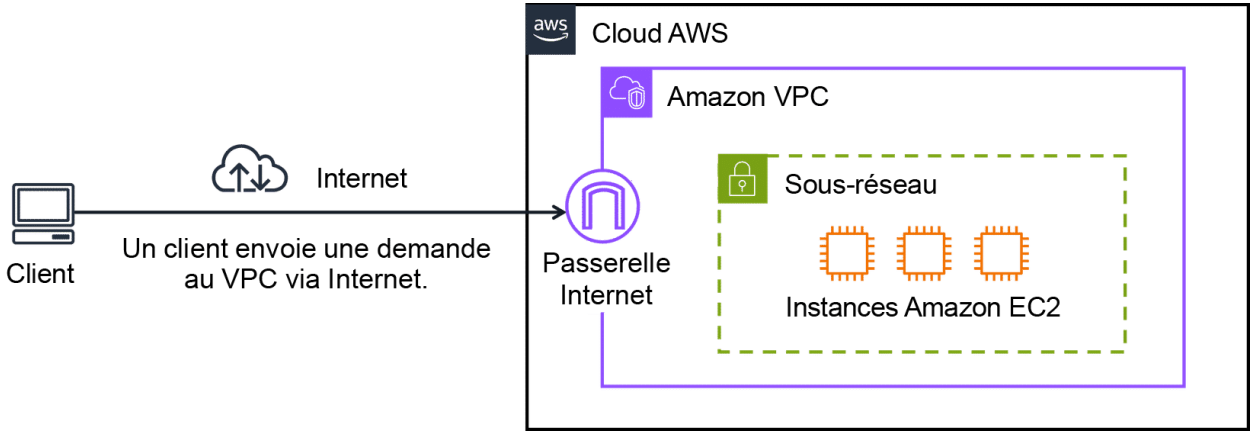
Amazon VPC

Avec Amazon VPC, vous pouvez allouer une section isolée du cloud AWS. Dans cette section isolée, vous pouvez lancer des ressources dans un réseau virtuel que vous définissez. Cela offre trois avantages principaux. Cela contribue à renforcer la sécurité, car vous pouvez sécuriser et surveiller les connexions, contrôler le trafic et restreindre l'accès aux instances. Amazon VPC vous donne un contrôle total sur le placement, la connectivité et la sécurité de vos ressources. La facilité d'utilisation d'Amazon VPC signifie que vous passerez moins de temps à configurer, gérer et valider votre réseau virtuel par rapport à la gestion de réseau sur site.

Sous-réseaux

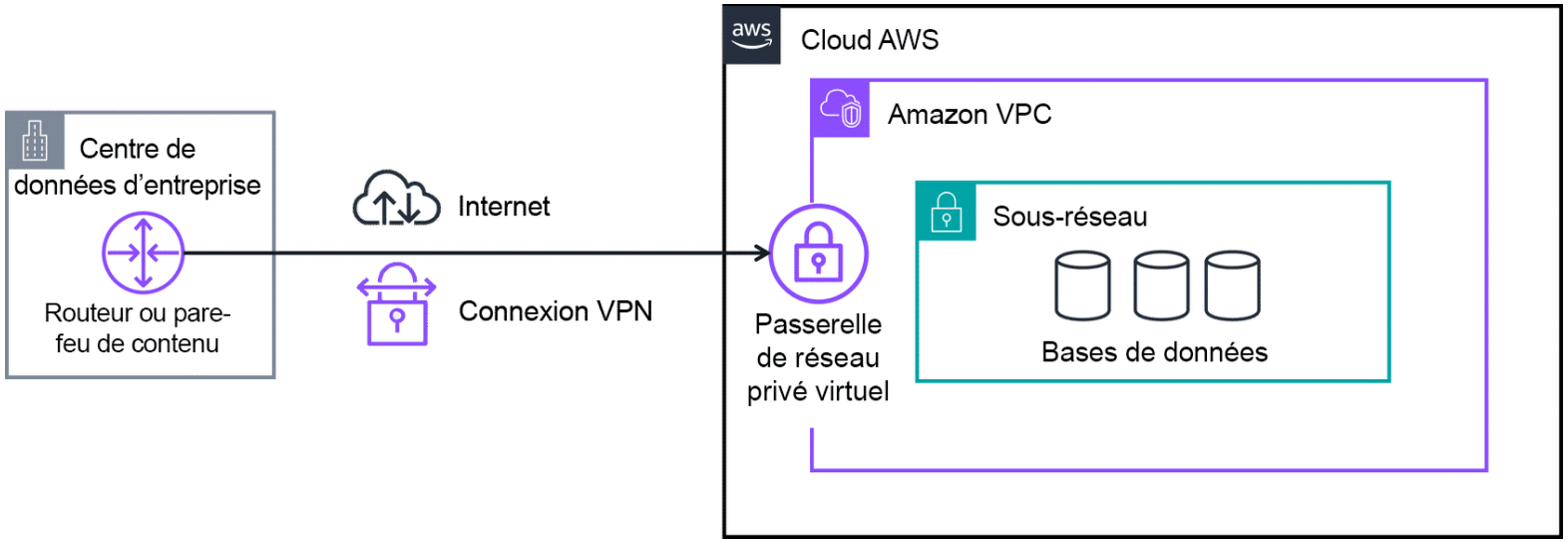
Au sein d'un Amazon VPC, vous pouvez organiser vos ressources en sous-sections ou sous-réseaux. Un sous-réseau est une section d'un Amazon VPC qui peut contenir des ressources, telles que des instances Amazon EC2.

Pour autoriser le trafic public à partir d'Internet à accéder à votre VPC, associez une **passerelle Internet** au VPC. Une passerelle Internet est une connexion entre un VPC et Internet. Vous pouvez voir une passerelle Internet comme la porte que les clients empruntent pour entrer dans le café. Sans passerelle Internet, personne ne peut accéder aux ressources de votre VPC.



Passerelle de réseau privé virtuel

Avec une passerelle de réseau privé virtuel, vous pouvez établir une connexion VPN entre votre VPC et un réseau privé, tel qu'un centre de données sur site ou un réseau d'entreprise interne. Une passerelle de réseau privé virtuel autorise le trafic vers le VPC uniquement s'il provient d'un réseau approuvé.



Stockage

Introduction à Amazon Storage

AWS propose de nombreuses solutions pour stocker, consulter, gérer et sauvegarder vos données dans le cloud. Ces systèmes de stockage virtuels éliminent le besoin de matériel physique dans votre centre de données tout en offrant la flexibilité nécessaire pour évoluer en fonction de vos besoins. AWS propose trois types distincts de stockage dans le cloud pour répondre aux différentes exigences et différents besoins des applications : le stockage par blocs, le stockage d'objets et le stockage de fichiers.

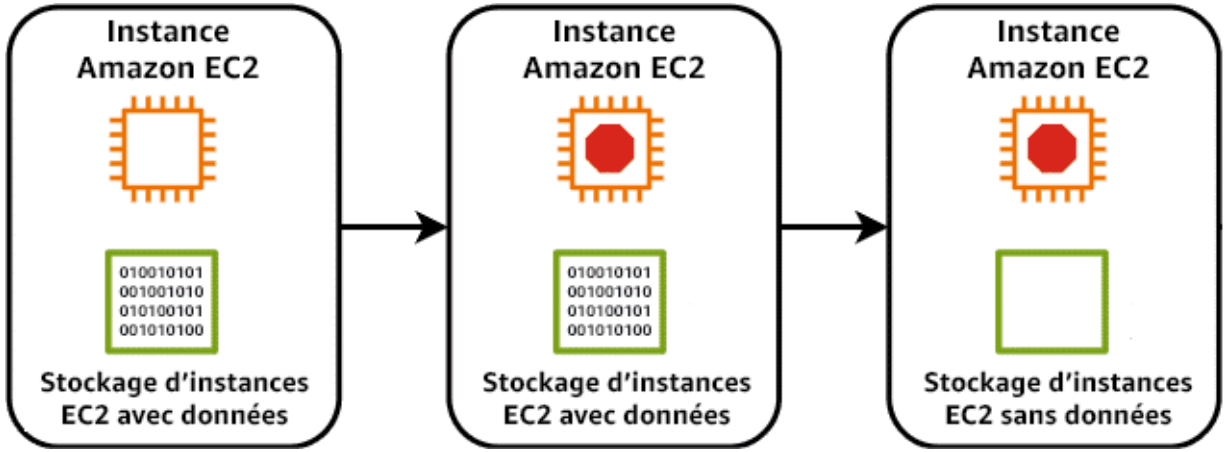
Stockage par blocs

Le stockage par blocs fournit des volumes de stockage persistants et à faible latence au niveau des blocs qui se connectent aux instances EC2, comme les disques durs physiques. Les volumes de stockage par blocs peuvent être chiffrés, sauvegardés via des instantanés et modifiés en cours d'utilisation sans perturber l'instance. AWS propose deux principaux services de stockage par blocs :

- *Stockage d'instance Amazon EC2*
  Un stockage par blocs non géré, non persistant et performant directement rattaché aux instances EC2 pour les données temporaires.
- *Amazon Elastic Block Store (EBS)*
  Un service géré qui fournit des volumes de stockage par blocs persistants pour les instances EC2, en proposant différents types pour différentes charges de travail

Stockage par blocs

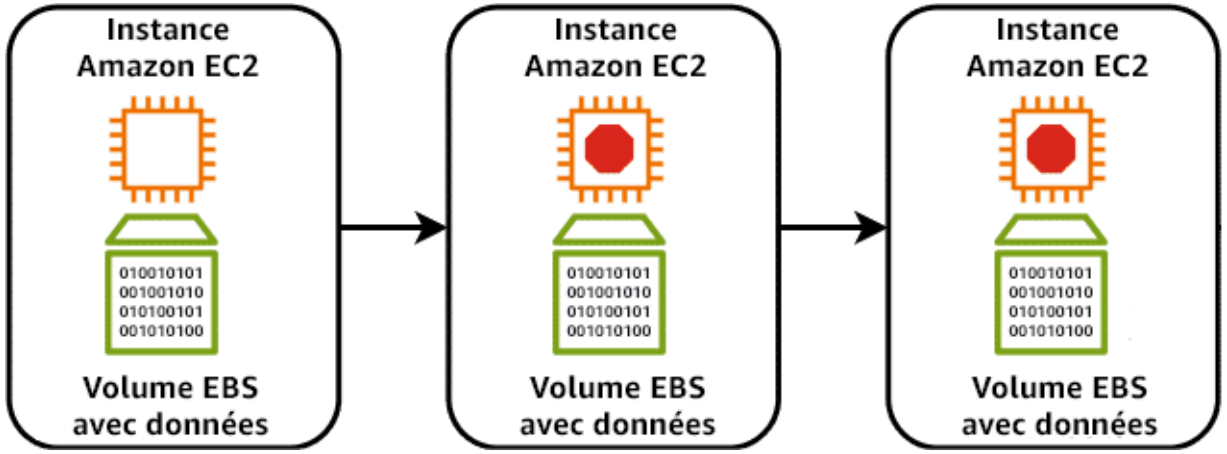
Le magasin d'instance Amazon EC2 n'est pas un service de stockage par blocs AWS autonome. Il fait plutôt référence au stockage par blocs qui est physiquement connecté à l'ordinateur hôte de l'instance EC2. Selon le type d'instance, le stockage d'instance EC2 peut être joint en tant que stockage par défaut. Étant donné que ses données sont perdues lorsqu'une instance est arrêtée ou interrompue, le stockage d'instance EC2 est idéal pour les besoins de stockage temporaire basés sur la mémoire, tels que les tampons, les caches et les données scratch. Il n'est pas recommandé pour les applications nécessitant la conservation de données.



Stockage par blocs

Amazon EBS fournit *des volumes de stockage par blocs permanents* à utiliser avec les instances Amazon EC2. Les volumes EBS agissent comme des disques durs externes, offrant des performances constantes et à faible latence pour des charges de travail telles que les bases de données et les systèmes de fichiers.

Les volumes EBS peuvent être facilement sauvegardés, redimensionnés et attachés à différentes instances EC2. Pour créer un volume EBS, vous devez définir la configuration pour des éléments tels que la taille et le type du volume. Une fois le volume créé, il peut être attaché à une instance Amazon EC2. Les volumes EBS étant destinés aux données qui doivent être conservées, il est important de les sauvegarder. Il est recommandé d'effectuer des sauvegardes incrémentielles des volumes EBS en créant des instantanés Amazon EBS.



Stockage d'objets

Le stockage d'objets est une architecture de stockage de données qui gère les données en tant qu'objets dans un espace d'adressage plat. Il offre une évolutivité illimitée, ce qui vous permet de stocker de grandes quantités de données non structurées sans vous soucier des contraintes de capacité. Le stockage d'objets fournit des fonctionnalités de métadonnées améliorées pour une gestion, une recherche et une analyse plus efficaces des données dans des jeux de données volumineux.

Le principal service de stockage d'objets AWS est le suivant :

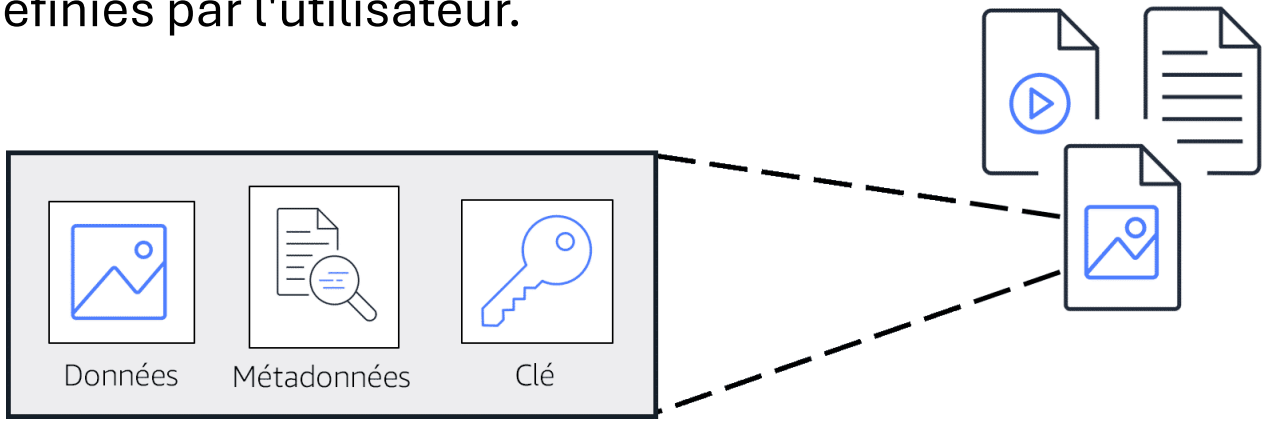
- *Amazon Simple Storage Service (S3)*
  Un service de stockage d'objets évolutif entièrement géré permettant de stocker et de récupérer n'importe quel volume de données, où que vous soyez.

Stockage d'objets : *Objets S3*

Dans Amazon S3, un objet est l'unité fondamentale du stockage des données. Lorsque vous chargez un fichier sur Amazon S3, il devient un objet et est stocké de manière durable dans plusieurs installations de la région que vous avez choisie.

Chaque objet inclut généralement les *données* mêmes, les *métadonnées* et un identifiant unique, ou *clé*. Les objets peuvent appartenir à n'importe quel type de fichier, comme des images, des vidéos, des documents ou des données d'application, et leur taille peut varier de quelques octets à plusieurs téraoctets.

Chaque objet Amazon S3 est identifié de manière unique dans un compartiment par sa clé, qui est essentiellement son nom de fichier. Les objets possèdent également des propriétés telles que l'ID de version, les informations de contrôle d'accès et les métadonnées définies par l'utilisateur.



Stockage d'objets : *Compartiments S3*

Un compartiment S3 est un conteneur permettant de stocker des objets dans Amazon S3. Les compartiments ont un nom unique au monde sur l'ensemble d'AWS, ce qui permet d'identifier et d'organiser vos données stockées. Les compartiments constituent l'unité de base pour le contrôle d'accès et peuvent contenir un nombre pratiquement illimité d'objets. Ils jouent un rôle crucial dans la gestion des données en permettant de regrouper des objets connexes et d'appliquer des politiques au niveau du compartiment.

Lorsque vous créez un compartiment, vous indiquez son nom et la région dans laquelle il résidera. Les compartiments peuvent être configurés avec différents paramètres, notamment la gestion des versions, la journalisation et les autorisations d'accès.

Stockage de fichiers

Les services de stockage de fichiers AWS fournissent des systèmes de fichiers partagés accessibles via des réseaux, afin que plusieurs utilisateurs et applications puissent accéder aux mêmes données simultanément. Ils offrent évolutivité et flexibilité afin que vous puissiez étendre la capacité de stockage à mesure que les besoins augmentent sans devoir gérer l'infrastructure physique. AWS propose deux services de stockage de fichiers principaux :

- *Amazon Elastic File System (EFS)*
  Un système de fichiers NFS évolutif et entièrement géré à utiliser avec les services cloud AWS et les ressources sur site.
- *Amazon FSx*
  Un service de stockage de fichiers entièrement géré pour les systèmes de fichiers courants tels que Windows, Lustre et NetApp ONTAP.

Services de stockage supplémentaires

Ces services ne correspondent pas parfaitement aux catégories que nous avons définies jusqu'à présent, mais il s'agit d'offres de stockage AWS importantes que vous devriez connaître.

- *AWS Storage Gateway*
  Un service de stockage sur le cloud hybride entièrement géré qui fournit un accès sur site à un stockage sur le cloud pratiquement illimité.
- *AWS Elastic Disaster Recovery*
  Un service entièrement géré qui rationalise la restauration de vos serveurs physiques, virtuels et basés sur le cloud dans AWS.

Amazon Elastic File System (EFS)

Amazon EFS est un service de stockage de fichiers évolutif et entièrement géré destiné à être utilisé avec les services cloud AWS et les ressources sur site. Il fonctionne à l'aide du *protocole NFS (Linux Network File System)* et *s'adapte automatiquement* aux pétaoctets lorsque vous ajoutez ou supprimez des fichiers sans perturber les applications.

EFS est conçu pour prendre en charge une grande variété de charges de travail et est accessible *simultanément par plusieurs instances EC2*.

Avantages d'Amazon EFS

Amazon EFS réplique automatiquement les données dans plusieurs zones de disponibilité d'une région pour garantir une haute disponibilité. Cette redondance intégrée protège contre les défaillances AZ et fournit un accès continu à vos systèmes de fichiers.

Amazon EFS prend en charge des milliers de connexions NFS simultanées, de sorte que plusieurs instances EC2 peuvent accéder simultanément au même système de fichiers. Ce modèle d'accès partagé rend EFS idéal pour les charges de travail collaboratives et les applications distribuées.

Amazon EFS augmente et diminue automatiquement au fur et à mesure que vous ajoutez et supprimez des fichiers, sans qu'il soit nécessaire de provisionner ou de gérer la capacité de stockage. Et comme vous ne payez que pour le stockage que vous utilisez, il s'agit d'une solution rentable pour répondre aux différentes exigences en matière de charge de travail.

Classes de stockage Amazon EFS

Les classes de stockage *EFS Standard* et *EFS Standard-Infrequent Access (Standard-IA)* offrent une résilience multi-AZ ainsi que les plus hauts niveaux de durabilité et de disponibilité. Ils ont un coût plus élevé en raison de leur disponibilité et de leur durabilité accrues.

Les solutions *EFS One Zone* et *EFS One Zone-Infrequent Access (EFS One Zone-IA)* permettent de réaliser des économies supplémentaires en enregistrant vos données dans une seule zone de disponibilité. En utilisant une seule zone de disponibilité, vous pouvez réduire vos coûts de stockage par rapport aux classes de stockage EFS standard.

La classe de stockage *EFS Archive* est optimisée en matière de coûts pour les données qui ne sont consultées que quelques fois par an ou moins et qui ne nécessitent pas les latences inférieures à la milliseconde d'EFS Standard. EFS Archive propose un prix de stockage jusqu'à 50 % inférieur à celui d'EFS Infrequent Access, offrant ainsi une expérience plus rentable pour les données froides et rarement consultées.

Cycle de vie des données Amazon EFS

Vous pouvez optimiser davantage les coûts de stockage Amazon EFS en déplaçant automatiquement les données entre les classes de stockage en fonction des modèles d'utilisation. Vous pouvez créer des politiques de cycle de vie qui déterminent quand et comment les fichiers passent d'un niveau de stockage à un autre. Ces politiques automatisées permettent de garantir que vos données se trouvent dans la classe de stockage la plus rentable, sans intervention manuelle.

AWS Storage Gateway

Storage Gateway est un service de stockage dans le cloud hybride qui permet d'intégrer de manière fluide les environnements sur site au stockage AWS Cloud. Vous pouvez l'utiliser pour étendre votre stockage local au cloud tout en maintenant un accès à faible latence aux données fréquemment utilisées.

Vous pouvez utiliser Storage Gateway pour rationaliser la gestion du stockage et réduire les coûts dans le cadre de cas d'utilisation pratiques du stockage dans le cloud hybride. Il s'agit notamment de déplacer les sauvegardes vers le cloud, d'utiliser des partages de fichiers sur site soutenus par un stockage dans le cloud et de fournir un accès à faible latence aux données dans AWS pour les applications sur site.

Passerelle de fichiers Amazon S3

La passerelle de fichiers Amazon S3 relie votre environnement local à Amazon S3. Il fournit aux applications sur site un accès à un stockage dans le cloud pratiquement illimité via des protocoles de fichiers familiers. La passerelle de fichiers S3 permet de stocker et de récupérer des objets du cloud à l'aide d'opérations de fichiers familières.

Lorsque vous déployez une passerelle de fichiers S3, il apparaît sur vos systèmes locaux en tant que serveur de fichiers standard. Les fichiers écrits sur ce serveur sont automatiquement chargés sur Amazon S3 tout en conservant un accès local aux données récemment utilisées grâce à une mise en cache intelligente. Cela signifie que vos applications peuvent continuer à utiliser les fichiers comme elles l'ont toujours fait, tandis que les données réelles sont stockées en toute sécurité dans le Cloud AWS.

Sauvegarde en volumes

Avec la sauvegarde en volumes, vous créez des volumes de stockage virtuels tout en conservant un accès local à vos données. Il fonctionne essentiellement comme une passerelle entre votre infrastructure sur site et le stockage Cloud AWS en présentant vos données cloud sous forme de volumes iSCSI qui peuvent être montés par vos applications existantes.

La sauvegarde en volumes fonctionne selon deux configurations principales :

- Le *mode volume mis en cache* stocke les données principales dans le cloud, tandis que les données fréquemment consultées sont mises en cache localement pour un accès à faible latence.
- Le *mode volume stocké* conserve localement l'intégralité de votre jeu de données tout en le sauvegardant de manière asynchrone dans le cloud sous forme d'instantanés EBS.

Sauvegarde sur bande virtuelle

La sauvegarde sur bande virtuelle permet de remplacer l'infrastructure de bandes physiques par des fonctionnalités de bandes virtuelles tout en bénéficiant de la durabilité et de l'évolutivité du stockage Cloud AWS. La sauvegarde sur bande virtuelle fournit une interface qui fonctionne avec les logiciels de sauvegarde sur bande existants, ce qui facilite la transition entre les bandes physiques et le stockage dans le cloud.

Lorsque vous déployez une sauvegarde sur bande virtuelle, elle se présente à vos applications de sauvegarde comme du matériel de bande standard. Votre logiciel de sauvegarde écrit les données sur ces bandes virtuelles comme il le ferait sur des bandes physiques et les enregistre dans Amazon S3. Vous pouvez également configurer la sauvegarde sur bande virtuelle pour transférer automatiquement les données moins fréquemment consultées vers une classe de stockage plus économique pour une conservation à long terme.

AWS Elastic Disaster Recovery

Elastic Disaster Recovery reproduit les charges de travail critiques vers AWS avec un temps d'immobilisation minimum. Les données au niveau des blocs de vos serveurs sont répliquées en permanence sur AWS, ce qui en fait la solution idéale pour les utilisations nécessitant des solutions robustes de reprise après sinistre. Il prend en charge à la fois les serveurs physiques et virtuels afin de permettre une reprise rapide en cas de panne, ce qui est particulièrement utile pour les secteurs tels que la santé, où la disponibilité du système est cruciale.

Vous pouvez utiliser Elastic Disaster Recovery pour réduire les temps d'immobilisation et les pertes de données tout en éliminant les coûts associés à la maintenance des centres de données secondaires. Il propose également des tests de reprise après sinistre sans interruption, ce qui signifie qu'il est capable de lancer rapidement des instances de reprise en cas de besoin.

AWS Elastic Disaster Recovery

Elastic Disaster Recovery reproduit les charges de travail critiques vers AWS avec un temps d'immobilisation minimum. Les données au niveau des blocs de vos serveurs sont répliquées en permanence sur AWS, ce qui en fait la solution idéale pour les utilisations nécessitant des solutions robustes de reprise après sinistre. Il prend en charge à la fois les serveurs physiques et virtuels afin de permettre une reprise rapide en cas de panne, ce qui est particulièrement utile pour les secteurs tels que la santé, où la disponibilité du système est cruciale.

Vous pouvez utiliser Elastic Disaster Recovery pour réduire les temps d'immobilisation et les pertes de données tout en éliminant les coûts associés à la maintenance des centres de données secondaires. Il propose également des tests de reprise après sinistre sans interruption, ce qui signifie qu'il est capable de lancer rapidement des instances de reprise en cas de besoin.

TP 3: Lambda 101

Mettre en place un pipeline **serverless event-driven** :
S3 (source) → Lambda (traitement) → S3 (destination) avec supervision.

Exigences fonctionnelles

- Quand un fichier image est uploadé dans le bucket source :
  - si c'est une image valide (.jpg, .jpeg, .png) → générer une miniature
  - sinon → ignorer proprement (log clair)
- Les miniatures vont dans le bucket destination
- La taille/format/préfixe sont **paramétrables par variables d'environnement**
- La fonction publie une **métrique CloudWatch personnalisée** "ImagesProcessed"
- Les logs doivent permettre de diagnostiquer facilement (succès / ignore / erreurs)

TP 3: Lambda 101

Variables d'environnement attendues

- DEST_BUCKET (obligatoire)
- THUMB_SIZE (défaut 200)
- OUTPUT_FORMAT (défaut JPEG) → JPEG ou PNG
- KEY_PREFIX (défaut thumb_)
- ALLOWED_EXTENSIONS (défaut .jpg,.jpeg,.png)
- METRIC_NAMESPACE (défaut INOW/Lambda101)

Livrables

- 2 buckets S3 (source + destination)
- 1 Lambda fonctionnelle (Python 3.12) + layer Pillow
- 1 event notification S3 **avec filtrage**
- Preuve : upload d'un .png → miniature créée ; upload d'un .txt → ignoré + log
- Courbe CloudWatch de la métrique **ImagesProcessed**

Bases de données

Introduction aux bases de données

AWS propose une large gamme de services de base de données pour répondre aux différents besoins de stockage et de gestion des données. Ces services sont conçus pour être évolutifs, fiables et faciles à utiliser pour les entreprises de toutes tailles. Les services de base de données AWS incluent des options pour les bases de données relationnelles, les bases de données non relationnelles, les caches en mémoire et des services spécialement conçus pour des cas d'utilisation tels que la gestion de documents.

Amazon Relational Database Service (Amazon RDS)

Amazon RDS est un service de base de données relationnelle *géré* qui traite les tâches de base de données de routine telles que les sauvegardes, les correctifs et la mise en service du matériel. Amazon RDS prend en charge plusieurs types de classes d'instance de base de données qui optimisent la mémoire, les performances ou les entrées/sorties (I/O).

Pour améliorer la résilience des données, Amazon RDS propose un déploiement multi-AZ et des sauvegardes automatisées, mais vous pouvez également créer manuellement des sauvegardes à l'aide d'instantanés de base de données. Il s'agit de sauvegardes complètes de l'intégralité de votre instance de base de données, qui peuvent être utiles à des fins de restauration ponctuelle ou d'archivage de données à long terme. Amazon RDS propose des fonctionnalités de sécurité, notamment l'isolation du réseau, le chiffrement en transit et le chiffrement au repos. Vous pouvez facilement dimensionner les ressources de base de données verticalement ou horizontalement selon vos besoins.

Moteurs de base de données pris en charge

Amazon RDS prend en charge différents moteurs de base de données, notamment Amazon Aurora, MySQL, PostgreSQL, Microsoft SQL Server, MariaDB et Oracle Database.

Amazon Relational Database Service (Amazon RDS)

Amazon RDS élimine les coûts initiaux élevés liés à l'achat et à la maintenance de l'infrastructure matérielle de base de données. Vous ne payez que pour les ressources de calcul et de stockage que vous consommez via un modèle flexible de paiement à l'utilisation. En tant que service géré, il réduit également les dépenses opérationnelles en automatisant les tâches administratives fastidieuses telles que les sauvegardes, les correctifs et la surveillance.

Amazon RDS améliore la fiabilité des bases de données grâce à des déploiements multi-AZ. Il réplique automatiquement les données vers une instance de secours située dans une zone de disponibilité différente. En cas de panne du système, de maintenance ou d'interruption de zone, Amazon RDS bascule automatiquement vers l'instance de secours sans intervention manuelle. Cela garantit un fonctionnement continu de la base de données avec un minimum d'interruption.

Amazon Relational Database Service (Amazon RDS)

Amazon RDS améliore les performances des bases de données grâce à une gestion automatisée des tâches d'allocation, de surveillance et d'optimisation des ressources. Il inclut des fonctionnalités telles que les sauvegardes automatisées et les répliques de lecture qui peuvent aider à décharger le trafic de lecture de l'instance principale. Les informations sur les performances d'Amazon RDS fournissent une surveillance et une analyse en temps réel de la charge de la base de données, afin de vous aider à identifier et à résoudre rapidement les problèmes de performances.

Amazon RDS améliore la sécurité des bases de données grâce à plusieurs niveaux de protection, notamment l'isolation des VPC ainsi que le chiffrement au repos et en transit. Il tire parti des sauvegardes automatisées et propose des déploiements multi-AZ pour garantir la résilience face aux défaillances potentielles du système.

Amazon Aurora

Amazon Aurora est une base de données relationnelle gérée conçue pour réduire les opérations d'I/O inutiles. Amazon Aurora est compatible avec MySQL et PostgreSQL, offre des performances et une disponibilité élevées, et s'adapte automatiquement à vos charges de travail. Amazon Aurora réplique les données dans plusieurs zones de disponibilité pour améliorer la durabilité et la tolérance aux pannes, et propose des sauvegardes automatisées, un chiffrement au repos et une surveillance continue.

Amazon Aurora

Aurora fournit un débit jusqu'à cinq fois supérieur à celui de MySQL standard et trois fois supérieur à celui de PostgreSQL. Aurora utilise un système de stockage distribué sur plusieurs nœuds pour fournir des performances et une disponibilité élevées.

Aurora augmente automatiquement la capacité de stockage de 10 Go à 128 To en fonction de votre utilisation réelle des données, ce qui élimine les conjectures lors de la planification des capacités. Aurora sauvegarde également en continu votre base de données sur Amazon Simple Storage Service (Amazon S3) afin de permettre une restauration instantanée.

Amazon Aurora réplique les données dans trois zones de disponibilité avec six copies de données et assure une disponibilité de 99,99 %. Aurora détecte automatiquement les défaillances des bases de données et redirige le trafic vers des répliques saines sans perte de données.

Bases de données NoSQL

Les bases de données NoSQL sont parfois appelées *bases de données non relationnelles*, car leurs structures sont différentes de celles des bases de données relationnelles comme Amazon RDS. Au lieu d'établir des relations entre lignes et colonnes, les bases de données NoSQL construisent une structure pour les données qu'elles contiennent à l'aide de *paires clé-valeur*. Avec les paires clé-valeur, les données sont organisées en éléments identifiés par des clés uniques.

Chaque clé possède un ou plusieurs attributs associés, ou valeurs, qui représentent diverses caractéristiques des données. Vous pouvez considérer une clé comme une entrée de mot dans un dictionnaire et la valeur comme sa définition associée. Tous les éléments du tableau ne doivent pas nécessairement avoir les mêmes attributs, et vous pouvez ajouter ou supprimer des attributs à tout moment.

Amazon DynamoDB

Amazon DynamoDB est un service de base de données NoSQL entièrement géré qui fournit des performances rapides et prévisibles pour les structures de données de documents et de valeurs clés. Il s'agit d'une option de base de données puissante et incroyablement rapide pour les cas d'utilisation nécessitant un schéma flexible. Elle est idéale pour les applications nécessitant des performances élevées et une évolutivité fluide.

Amazon DynamoDB s'adapte parfaitement à vos données sans affecter les performances, ce qui signifie que vous ne payez que pour les ressources que vous utilisez. Il inclut également des fonctionnalités de sécurité intégrées pour une meilleure protection et répartit automatiquement vos données sur plusieurs serveurs pour gérer votre charge de travail.

Amazon DynamoDB

Amazon DynamoDB augmente ou diminue automatiquement le débit en fonction de l'utilisation réelle, ce qui garantit des performances constantes sans intervention manuelle. Vous pouvez spécifier des niveaux d'utilisation cibles et Amazon DynamoDB alloue automatiquement la capacité nécessaire pour maintenir ces objectifs. Sans aucune limite pratique quant à la taille des tables ou à la quantité de données stockées, Amazon DynamoDB peut parfaitement s'adapter à des applications en plein développement.

Amazon DynamoDB offre des temps de réponse de l'ordre de quelques millisecondes à n'importe quelle échelle, ce qui en fait la solution idéale pour les applications hautes performances. Il maintient des performances constantes en distribuant automatiquement les données sur plusieurs serveurs et SSD.

Amazon DynamoDB

Amazon DynamoDB assure une disponibilité des données de 99,999 % en les répliquant sur trois sites distincts au sein de chaque région AWS. Il conserve également plusieurs copies dans des régions AWS distinctes, afin de garantir une tolérance aux pannes et une durabilité des données intégrées. Cela garantit un fonctionnement continu et une protection contre la perte de données, même en cas de défaillance de certaines installations.

Amazon DynamoDB propose des fonctionnalités de chiffrement complètes pour protéger les informations au repos et en transit. Toutes les données sont automatiquement chiffrées en arrière-plan avant d'être écrites sur la couche de stockage. Amazon DynamoDB offre la possibilité de choisir entre différents types de clés de chiffrement pour un contrôle de sécurité personnalisé.

Amazon ElastiCache

Amazon ElastiCache est un service de mise en cache en mémoire entièrement géré qui a été conçu pour réduire la complexité de l'administration des systèmes de mise en cache en mémoire. Cela signifie que vous pouvez continuer à utiliser les mêmes outils et configurations Redis, Valkey ou Memcached pour dimensionner vos charges de travail. Il détecte et remplace automatiquement les nœuds défaillants, ce qui le rend idéal pour les applications nécessitant des performances élevées et constantes.

Amazon ElastiCache

Amazon ElastiCache rationalise le déploiement et la maintenance des environnements de mise en cache en mémoire, offrant une haute disponibilité pour Redis, Valkey et Memcached en gérant automatiquement la mise en service du matériel, l'application de correctifs logiciels et la surveillance. Amazon ElastiCache offre une évolutivité fluide qui vous permet d'ajouter ou de supprimer des nœuds en fonction de l'évolution de la demande.

Amazon ElastiCache assure une haute disponibilité en surveillant en permanence les nœuds principaux pour détecter d'éventuelles défaillances. Lorsque des problèmes sont détectés, il maintient la disponibilité des applications tout en promouvant un nœud de réplication pour qu'il devienne le nouveau nœud principal sans intervention manuelle. L'ensemble du processus de restauration se termine généralement en quelques minutes, ce qui minimise les temps d'arrêt et préserve les opérations en cas de perturbations de l'infrastructure.

Amazon ElastiCache

Amazon ElastiCache permet une réplication automatique sur plusieurs zones de disponibilité afin de vous protéger contre les défaillances de l'infrastructure. Vous pouvez configurer des nœuds principaux et des nœuds de réplication dans différentes zones de disponibilité en fonction de leurs exigences de durabilité. Cela permet de garantir que les données restent accessibles même en cas de panne d'une zone.

Amazon ElastiCache prend en charge des mécanismes de chiffrement des données afin de protéger les informations sensibles tout au long de leur cycle de vie. Le chiffrement au repos protège les données lorsqu'elles sont stockées sur disque et lors de sauvegardes automatisées. Le chiffrement en transit sécurise les données circulant entre les clients et les nœuds de cache en utilisant la sécurité de la couche de transport, ou TLS, pour les connexions chiffrées.

TP Lambda 102 — Traitement de commandes e-commerce

Résultat attendu

En déposant un fichier orders/ORD-2026-001.json dans S3 :

- une Lambda **valide** la commande et l'envoie en SQS
- une Lambda **worker** consomme la file, **enregistre** la commande dans DynamoDB
- les erreurs vont en **DLQ**
- DynamoDB contient les statuts (VALIDATED, PROCESSED, REJECTED, DUPLICATE)
- Métrique CloudWatch OrdersProcessed, OrdersRejected, OrdersDuplicate

TP Lambda 102 — Traitement de commandes e-commerce

Résultat attendu

En déposant un fichier orders/ORD-2026-001.json dans S3 :

- une Lambda **valide** la commande et l'envoie en SQS
- une Lambda **worker** consomme la file, **enregistre** la commande dans DynamoDB
- les erreurs vont en **DLQ**
- DynamoDB contient les statuts (VALIDATED, PROCESSED, REJECTED, DUPLICATE)
- Métrique CloudWatch OrdersProcessed, OrdersRejected, OrdersDuplicate

TP Lambda 102 — Traitement de commandes e-commerce

Contexte

Vous travaillez chez **INOW-Shop**. Le site exporte des commandes sous forme de fichiers JSON déposés dans un bucket S3.
Vous devez mettre en place un traitement **asynchrone, découplé et résilient**.

Objectifs :

- Déclencher une Lambda à l'upload d'un JSON dans S3
- Valider un payload de commande (champ obligatoires + règles simples)
- Publier un message dans une file **SQS**
- Consommer la file avec une Lambda worker
- Enregistrer la commande et son statut dans **DynamoDB**
- Gérer les erreurs : **retries + DLQ**
- Mettre en place une logique **idempotente** (pas de double traitement)

TP Lambda 102 — Traitement de commandes e-commerce

Contraintes

- Région : eu-west-3
- Runtime : Python 3.12 (ou 3.9 si tu standardises autrement)
- Messages SQS en JSON
- DynamoDB en PAY_PER_REQUEST

Scénarios de validation

- Upload d'une commande valide
- Upload d'une commande invalide (ex: amount négatif)
- Upload deux fois la même commande (idempotence)

Livrables

- Bucket S3 + prefix orders/
- SQS main + DLQ
- DynamoDB table tp-orders
- 2 Lambdas :
  tp-order-validator
  tp-order-worker
- Tests prouvant :
  commande OK → DynamoDB PROCESSED
  commande invalide → DLQ + DynamoDB REJECTED
  doublon (même order_id) → DynamoDB DUPLICATE (ou ignoré)

TP Lambda 102 — Traitement de commandes e-commerce

Option : Mission CI/CD

- Créer un repo **AWS CodeCommit** tp-orders-lambda-102
- Y pousser le code validator et worker + un buildspec.yml
- Créer un pipeline **CodePipeline** :
  - Source : CodeCommit (branch main)
  - Build : CodeBuild (packaging zip)
  - Deploy : réalisé dans CodeBuild (commande aws lambda update-function-code)
- Valider : un commit déclenche le pipeline et met à jour la Lambda.

TP 1: Architecture scalable avec AWS EC2, Auto Scaling Group et Elastic Load Balancer

1. Architecture Cible:

```
Internet
   |
   ▼
[ Application Load Balancer ] (HTTP:80)
   |
   ▼
[ Target Group (HTTP) ] ----> [ Auto Scaling Group ]
                                  |   \
                                  |    \
                                 EC2    EC2   (2+ AZ)
```

TP 1: Architecture scalable avec AWS EC2, Auto Scaling Group et Elastic Load Balancer

2. Préparation du compte et vérifications

- **Étapes :**
- Connectez-vous à la console AWS (IAM User).
- En haut à droite, sélectionnez la région **Paris (eu-west-3)**.
- Vérifiez que le **VPC par défaut** existe :
  - Menu : *VPC → Your VPCs → default*
  - Il doit contenir **3 subnets** (chacun dans une AZ différente).

TP 1: Architecture scalable avec AWS EC2, Auto Scaling Group et Elastic Load Balancer

3. Clé SSH et Groupes de sécurité

Objectif :

- Permettre un accès sécurisé aux instances et définir les flux autorisés.

Étapes :

3.1 Créer une paire de clés

- *EC2 → Key Pairs → Create Key Pair*
  - Name : tp-ec2-key
  - Type : RSA, Format : .pem
  - Télécharger et conserver le fichier.

3.2 Créer deux Security Groups

- **SG-ALB** (Load Balancer)
  - Autoriser HTTP port **80** depuis **toute adresse (0.0.0.0/0)**
  - Sortant : tout autorisé
- **SG-EC2** (Instances)
  - Autoriser HTTP port **80** depuis **SG-ALB** uniquement
  - Autoriser SSH port **22** depuis **votre IP publique uniquement**
  - Sortant : tout autorisé

TP 1: Architecture scalable avec AWS EC2, Auto Scaling Group et Elastic Load Balancer

3. Clé SSH et Groupes de sécurité

Objectif :

- Permettre un accès sécurisé aux instances et définir les flux autorisés.

Étapes :

3.1 Créer une paire de clés

- *EC2 → Key Pairs → Create Key Pair*
  - Name : tp-ec2-key
  - Type : RSA, Format : .pem
  - Télécharger et conserver le fichier.

3.2 Créer deux Security Groups

- **SG-ALB** (Load Balancer)
  - Autoriser HTTP port **80** depuis **toute adresse (0.0.0.0/0)**
  - Sortant : tout autorisé
- **SG-EC2** (Instances)
  - Autoriser HTTP port **80** depuis **SG-ALB** uniquement
  - Autoriser SSH port **22** depuis **votre IP publique uniquement**
  - Sortant : tout autorisé

TP 1: Architecture scalable avec AWS EC2, Auto Scaling Group et Elastic Load Balancer

4. Création du Launch Template

Objectif :

- Standardiser la configuration de nos instances EC2 (AMI, type, sécurité, User Data…).

Étapes :

- *EC2 → Launch Templates → Create Launch Template*
  - Name : tp-lt-web
  - AMI : Amazon Linux 2023 (x86_64)
  - Instance type : t3.micro
  - Key pair : tp-ec2-key
  - Security group : SG-EC2
  - User Data : (coller ce script)

```
#!/bin/bash
dnf -y update
dnf -y install nginx
INSTANCE_ID=$(curl -s http://169.254.169.254/latest/meta-data/instance-id)
echo "<h1>TP EC2/ASG/ALB</h1><p>Instance: ${INSTANCE_ID}</p>" > /usr/share/nginx/html/index.html
systemctl enable nginx
systemctl start nginx
```

TP 1: Architecture scalable avec AWS EC2, Auto Scaling Group et Elastic Load Balancer

5. Création du Target Group

Objectif :

- Définir les cibles (instances EC2) qui recevront le trafic de l'ALB.

Étapes :

- *EC2 → Target Groups → Create Target Group*
  - Type : Instances
  - Name : tp-tg-web
  - Protocol : HTTP, Port 80
  - VPC : default
  - Health check path : /
  - Healthy threshold : 2
  - Unhealthy threshold : 2

TP 1: Architecture scalable avec AWS EC2, Auto Scaling Group et Elastic Load Balancer

6. Création du Load Balancer (ALB)

Étapes :

- *EC2 → Load Balancers → Create Load Balancer → Application Load Balancer*
  - Name : tp-alb-web
  - Scheme : Internet-facing
  - IP type : IPv4
  - VPC : default
  - Subnets : choisir au moins **2 AZ**
  - Security Group : SG-ALB
  - Listener : HTTP:80 → Forward vers tp-tg-web

TP 1: Architecture scalable avec AWS EC2, Auto Scaling Group et Elastic Load Balancer

7. Création du Auto Scaling Group

Étapes :

- *EC2 → Auto Scaling Groups → Create ASG*
  - Launch Template : tp-lt-web
  - VPC : default
  - Subnets : 2 ou 3 zones de disponibilité
  - Target Group : tp-tg-web
  - Desired Capacity : 2
  - Min : 2
  - Max : 4
  - Health check : cocher “ELB” (grace period 300s)

TP 1: Architecture scalable avec AWS EC2, Auto Scaling Group et Elastic Load Balancer

8. Ajout d'une politique de scaling automatique

Étapes :

- Dans votre ASG → onglet *Automatic scaling* → *Add policy*
  - Type : Target Tracking
  - Metric : Average CPU Utilization
  - Target value : 50%

9. Vérification du déploiement

Étapes :

- Attendre que les instances EC2 soient **Healthy** dans le Target Group.
- Copier le **DNS Name** du Load Balancer (ex: tp-alb-web-123.eu-west-3.elb.amazonaws.com).
- Ouvrir dans le navigateur.

Résultat attendu :

Vous voyez une page HTML :
TP EC2/ASG/ALB - Instance: i-xxxxxxxxx

TP 1: Architecture scalable avec AWS EC2, Auto Scaling Group et Elastic Load Balancer

10. Simulation de charge et scaling automatique

Étapes :

- Depuis **CloudShell** :

```
sudo dnf -y install httpd-tools
ab -n 100000 -c 200 http://<ALB_DNS>/
```

11. Instance Refresh (rolling update)

Étapes :

- Dupliquer le Launch Template (nouvelle version)
- Modifier le script User Data pour changer le contenu HTML (ex: version 2 du site)
- Dans l'ASG → onglet *Instance Refresh* → Lancer un refresh

TP 1: Architecture scalable avec AWS EC2, Auto Scaling Group et Elastic Load Balancer

12. Test d'auto-healing

Étapes :

- Supprimez manuellement une instance EC2 depuis la console.
- Attendez 1-2 minutes.

Résultat :

- L'ASG recrée automatiquement une nouvelle instance pour maintenir la capacité.

13. Supervision

Étapes :

- *CloudWatch → Metrics → EC2/ASG/ELB*
- Observer CPU, Network, RequestCount, etc.

TP 1: Architecture scalable avec AWS EC2, Auto Scaling Group et Elastic Load Balancer

14. Nettoyage complet

- Ordre de suppression :
- ASG (cocher “Delete all instances”)
- Listener + ALB
- Target Group
- Launch Template
- Security Groups
- Key Pair

TP 1: Architecture scalable avec AWS EC2, Auto Scaling Group et Elastic Load Balancer

Pour aller plus loin

- Ajouter un **certificat HTTPS** via **AWS Certificate Manager (ACM)**
- Activer **access logs** du Load Balancer dans S3
- Créer une **CloudWatch Alarm** pour surveiller les erreurs HTTP 5xx
- Déployer la même architecture en **Terraform**

TP 2: Réseaux AWS (VPC, Subnets, Routage, Sécurité)

Architecture Cible:

```
                        +--------------------------+
                        |        Internet          |
                        +--------------------------+
                                    |
                           [ Internet Gateway ]
                                    |
                                    |
          +-----------------------------------------------+
          |                  VPC 10.0.0.0/16              |
          |                                               |
+-------------------------+                +-------------------------+
| Public subnet (10.0.1.0/24)              | Private subnet (10.0.2.0/24)
| AZ a                                   | AZ a
| [ Bastion EC2 (SSH) ]                  | [ Private EC2 / Web / DB ]
|  SG : SSH open from home               |  SG : Only inbound from Bastion
+-------------------------+              +-------------------------+
         |                                            |
     [ Route table: default route → IGW ]         [ Route table: default route → NAT GW ]
```

TP 2: Réseaux AWS (VPC, Subnets, Routage, Sécurité)

Étape 1 — Créer un VPC

Objectif

- Créer un VPC isolé avec une plage IP adaptée.

Étapes

- *VPC → Your VPCs → Create VPC*
  - Name: tp-vpc
  - CIDR: 10.0.0.0/16
  - Tenancy: default
  - DNS hostnames: **enabled**

TP 2: Réseaux AWS (VPC, Subnets, Routage, Sécurité)

Étape 2 — Créer deux subnets (public et privé)

Étapes

- *VPC → Subnets → Create subnet*
  - VPC: tp-vpc
  - Subnet 1: tp-subnet-public → 10.0.1.0/24 (AZ: eu-west-3a)
  - Subnet 2: tp-subnet-private → 10.0.2.0/24 (AZ: eu-west-3a)

Étape 3 — Créer et attacher un Internet Gateway (IGW)

Étapes

- *VPC → Internet Gateways → Create Internet Gateway*
  - Name: tp-igw
- *Actions → Attach to VPC → tp-vpc*

TP 2: Réseaux AWS (VPC, Subnets, Routage, Sécurité)

Étape 4 — Tables de routage

Étapes

- *VPC → Route Tables → Create route table*
  - Name: tp-public-rt
  - VPC: tp-vpc
- Ajouter une route :
  - Destination: 0.0.0.0/0
  - Target: tp-igw
- Associer à tp-subnet-public
- Créer une 2e table :
  - Name: tp-private-rt
  - VPC: tp-vpc
  - (aucune route vers IGW pour le moment)
  - Associer à tp-subnet-private

TP 2: Réseaux AWS (VPC, Subnets, Routage, Sécurité)

Étape 5 — Créer un NAT Gateway (pour les subnets privés)

Étapes

- *VPC → NAT Gateways → Create*
  - Subnet: tp-subnet-public
  - Elastic IP: Allocate new
- Une fois créé, ajouter une route dans tp-private-rt :
  - Destination: 0.0.0.0/0
  - Target: ce NAT Gateway.

TP 2: Réseaux AWS (VPC, Subnets, Routage, Sécurité)

Étape 6 — Groupes de sécurité et NACLs

6.1 Security Groups

- **SG-Bastion**
  - Inbound: SSH (22) depuis votre IP publique
- **SG-Private**
  - Inbound: SSH (22) depuis **SG-Bastion**

6.2 Network ACLs (optionnel)

- Tester la différence entre SG et NACL :
  - Créer une NACL stricte pour le subnet privé (deny tout, sauf port 22 depuis le subnet public)

TP 2: Réseaux AWS (VPC, Subnets, Routage, Sécurité)

Étape 7 — Déployer les instances

Étapes

- **Bastion (public)**
  - Launch Template : Amazon Linux 2023
  - Subnet : tp-subnet-public
  - SG : SG-Bastion
  - Key Pair : votre clé
  - IP publique : **enable**
- **Web (privé)**
  - Même AMI
  - Subnet : tp-subnet-private
  - SG : SG-Private
  - IP publique : **disable**

Test

- Connectez-vous au Bastion :
  ssh -i tp-ec2-key.pem ec2-user@<Bastion-IP>
- Depuis le bastion :
  ssh ec2-user@<Private-Instance-IP>

TP 2: Réseaux AWS (VPC, Subnets, Routage, Sécurité)

Étape 8 — Vérifications et diagnostic

Test	Commande	Résultat attendu
Ping interne privé → bastion	ping 10.0.1.x	OK
Ping Internet depuis bastion	ping 8.8.8.8	OK
Ping Internet depuis instance privée	ping 8.8.8.8	OK (via NAT)
SSH direct depuis Internet vers privé	KO	Bloqué

TP 2: Réseaux AWS (VPC, Subnets, Routage, Sécurité)

Étape 9 — Observation dans VPC Flow Logs (bonus)

- *VPC → Flow Logs → Create flow log*
  - Resource: tp-vpc
  - Destination: CloudWatch Logs
- Lancer un curl ou ping depuis vos instances
- Observer les flux dans CloudWatch Logs

Étape 10 — Nettoyage

- Supprimer : instances, NAT Gateway, Elastic IP, VPC (et ses sous-ressources)

Mini TP 1: Lambda 101 “Redimensionneur d’images S3”

Architecture simplifiée

```
[Utilisateur]
   ↓ (Upload image)
[S3 Bucket source]
   ↓ (Event trigger)
[Lambda Function]
   ↓ (Résultat)
[S3 Bucket de destination : images redimensionnées]
```

Fonctionnement

Un utilisateur charge une image dans le bucket S3 source
S3 envoie un **événement** à Lambda
La fonction Lambda traite l’image (ex: redimensionnement avec Pillow)
L’image traitée est sauvegardée dans le bucket destination

Ressources AWS

S3 (stockage source et destination)
Lambda (fonction serverless Python/Node.js)
IAM Role (permissions : s3:GetObject, s3:PutObject, logs:*)
CloudWatch Logs (logs d’exécution de la fonction)

Mini TP 1: Lambda 101 “Redimensionneur d’images S3”

Étape 1 : Préparation S3

- Crée deux buckets :
  - live-<prenom>-images-source
  - live-<prenom>-images-thumb
- Active les événements pour le bucket source :
  - Type : **PUT (ObjectCreated)**
  - Cible : **Fonction Lambda (à venir)**

Mini TP 1: Lambda 101 “Redimensionneur d’images S3”

Lambda Layer (recommandé, propre et réutilisable)

- Compatible avec Python 3.12 (runtime Lambda basé sur **Amazon Linux 2023**).
- Prépare le layer en local (Linux/macOS) :

```
mkdir -p layer/python
pip install --upgrade pip
pip install "Pillow==10.*" -t layer/python
cd layer && zip -r ../pillow-layer.zip python && cd ..
```

- Console AWS → **Lambda → Layers → Create layer**

Name: pillow-py312

Upload: pillow-layer.zip

Runtimes: **Python 3.12**

Architectures: celles de ta fonction (x86_64 et/ou arm64)

Sur ta **fonction Lambda** → **Configuration → Layers → Add a layer** → sélectionne pillow-py312.

Mini TP 1: Lambda 101 “Redimensionneur d’images S3”

Étape 2 : Créer la fonction Lambda

AWS → **Lambda → Create function**

Name : live-lambda-picture
Runtime : Python 3.12 (ou Node.js 20)
Permissions : **Create a new role with basic permissions**

Ajouter le code suivant :

```
import boto3
from PIL import Image
import io
s3 = boto3.client('s3')
dest_bucket = 'live-<prenom>-images-thumb'
def lambda_handler(event, context):
  src_bucket = event['Records'][0]['s3']['bucket']['name']
  key = event['Records'][0]['s3']['object']['key']
  image = s3.get_object(Bucket=src_bucket, Key=key)['Body'].read()
  img = Image.open(io.BytesIO(image))
  img.thumbnail((200, 200))
  buffer = io.BytesIO()
  img.save(buffer, 'JPEG')
  buffer.seek(0)
  s3.put_object(Bucket=dest_bucket, Key=f"thumb_{key}", Body=buffer, ContentType='image/jpeg')
  return {"status": "OK", "image": key}
```

Mini TP 1: Lambda 101 “Redimensionneur d’images S3”

Étape 3 : Configurations Lambda

- **Handler** : lambda_function.lambda_handler
- **Memory** : 512 MB
- **Timeout** : 1 minute
- **Variable d’environnement :**

DEST_BUCKET=live-<prenom>-images-thumb

- **Rôle IAM** :
  - s3:GetObject
  - s3:PutObject
  - logs:CreateLogGroup
  - logs:CreateLogStream
  - logs:PutLogEvents

Mini TP 1: Lambda 101 “Redimensionneur d’images S3”

Étape 4 : Test

- Téléverse une image dans le bucket live-<prenom>-images-source
- Vérifie :
  - Le bucket live-<prenom>-images-thumb contient une image thumb_<nom>.jpg
  - Les logs CloudWatch montrent :
  - status: OK, image: <nom_de_l’image>

Mini TP 1: Lambda 101 “Redimensionneur d’images S3”

IAM & Nettoyage

Permissions IAM minimales :

```
{
  "Action": [
    "s3:GetObject",
    "s3:PutObject",
    "logs:CreateLogGroup",
    "logs:CreateLogStream",
    "logs:PutLogEvents"
  ],
  "Effect": "Allow",
  "Resource": "*"
}
```

Nettoyage :

Supprimer les deux buckets S3

Supprimer la fonction Lambda

Supprimer le rôle IAM associé

TP Global – Déploiement d’une architecture web 3-tiers sur AWS

Objectifs

- Mettre en œuvre une architecture complète sur AWS simulant un **site web d’entreprise** :
- Front-end hébergé sur **S3 + CloudFront**
- Back-end applicatif sur **EC2** dans un **VPC** sécurisé
- Base de données **RDS**
- Supervision avec **CloudWatch**
- Contrôle d’accès via **IAM**
- Optimisation des coûts via **Cost Explorer / Pricing Calculator**

Mise en situation

- Vous êtes **ingénieur cloud** pour une PME appelée **INOW-Store**.
  Elle souhaite héberger un site e-commerce sur AWS avec :
- Une page web statique publique (hébergée sur S3)
- Une API applicative (sur EC2)
- Une base de données MySQL managée (RDS)
- Une surveillance et une estimation de coût mensuel
- Vous devez construire cette architecture **à la main** (console AWS) et **via la CLI**.

TP Global – Déploiement d'une architecture web 3-tiers sur AWS

Architecture cible à reproduire

```
              +-----------------------+
              |     CloudFront        |
              +----------+------------+
                         |
              Static Website (S3)
                         |
          +------------+-------------+
          |     VPC 10.0.0.0/16      |
          +------------+-------------+
                         |
+--------------------+---------------------+
| Public Subnet 10.0.1.0/24 (AZ-a)         |
| EC2 Instance (Web/API)                   |
+------------------------------------------+
| Private Subnet 10.0.2.0/24 (AZ-b)        |
| RDS MySQL Database                       |
+------------------------------------------+
                         |
                IAM Roles & Security
                         |
            CloudWatch Monitoring
```

TP Global – Déploiement d’une architecture web 3-tiers sur AWS

1. Configuration initiale

- **Étape 1.1 – Configurer la CLI**

```
aws configure

Region : eu-west-3
Format : json
```

- **Étape 1.2 – Vérifier les identités**

aws sts get-caller-identity

- **Étape 1.3 – Créer un utilisateur IAM (via console)**

Ouvrir **IAM → Users → Add user**

Nom : inow-admin

Permissions : *AdministratorAccess*

Cocher *Access key* et *Console password*

Télécharger le .csv des credentials

💡 *Correction CLI équivalente :*

aws iam create-user --user-name inow-admin

aws iam attach-user-policy --user-name inow-admin --policy-arn arn:aws:iam::aws:policy/AdministratorAccess

aws iam create-login-profile --user-name inow-admin --password 'Inow@2024'

TP Global – Déploiement d'une architecture web 3-tiers sur AWS

2. Réseau (VPC + Subnets)

- **Étape 2.1 – Créer le VPC**

aws ec2 create-vpc --cidr-block 10.0.0.0/16

- **Étape 2.2 – Créer les sous-réseaux**

aws ec2 create-subnet --vpc-id <vpc-id> --cidr-block 10.0.1.0/24 --availability-zone eu-west-3a --map-public-ip-on-launch

aws ec2 create-subnet --vpc-id <vpc-id> --cidr-block 10.0.2.0/24 --availability-zone eu-west-3b

aws ec2 create-subnet --vpc-id <vpc-id> --cidr-block 10.0.3.0/24 --availability-zone eu-west-3c

- **Étape 2.3 – Créer l'Internet Gateway**

aws ec2 create-internet-gateway

aws ec2 attach-internet-gateway --internet-gateway-id <igw-id> --vpc-id <vpc-id>

- **Étape 2.4 – Créer la table de routage**

aws ec2 create-route-table --vpc-id <vpc-id>

aws ec2 create-route --route-table-id <rtb-id> --destination-cidr-block 0.0.0.0/0 --gateway-id <igw-id>

aws ec2 associate-route-table --subnet-id <subnet-public-id> --route-table-id <rtb-id>

TP Global – Déploiement d'une architecture web 3-tiers sur AWS

3. Instance EC2 (Serveur web/API)

- **Étape 3.1 – Créer la paire de clés**

aws ec2 create-key-pair --key-name inow-key --query 'KeyMaterial' --output text > inow-key.pem

chmod 400 inow-key.pem

- **Étape 3.2 – Créer le Security Group**

aws ec2 create-security-group --group-name inow-sg --description "Web access" --vpc-id <vpc-id>

aws ec2 authorize-security-group-ingress --group-id <id-group> --protocol tcp --port 22 --cidr 0.0.0.0/0

aws ec2 authorize-security-group-ingress --group-id <id-group> --protocol tcp --port 80 --cidr 0.0.0.0/0

- **Étape 3.3 – Lancer une instance**

```
aws ec2 run-instances \
  --image-id ami-0fa91bc90632c73c9 \
  --instance-type t2.micro \
  --key-name inow-key \
  --security-groups inow-sg \
  --subnet-id <subnet-public-id>
```

- **Étape 3.4 – Connexion et installation**

ssh -i inow-key.pem ubuntu@<ip_publique>

sudo apt update && sudo apt install -y nginx

echo "INOW Store - API en ligne" | sudo tee /var/www/html/index.html

sudo systemctl start nginx

✅ Vérification via navigateur : http://<ip_publique>

TP Global – Déploiement d’une architecture web 3-tiers sur AWS

4. Front-end statique sur S3 + CloudFront

- **Étape 4.1 – Créer le bucket**

aws s3 mb s3://inow-store-<votre-nom>

- **Étape 4.2 – Uploader le site**

echo "<h1>Bienvenue sur INOW Store</h1>" > index.html

aws s3 cp index.html s3://inow-store-<votre-nom>/ --acl public-read

- **Étape 4.3 – Activer l’hébergement statique (console)**

S3 → Propriétés → Hébergement statique → Activer

Page d’index : index.html

- **Étape 4.4 – Créer la distribution CloudFront**

Source : votre bucket S3

Origin access : public

Cache : par défaut

💡 *Option CLI :*

aws cloudfront create-distribution --origin-domain-name inow-store-<votre-nom>.s3.amazonaws.com

TP Global – Déploiement d’une architecture web 3-tiers sur AWS

5. Base de données RDS

- **Étape 5.1 – Créer une base MySQL (console)**

Service : RDS → Create Database

Engine : MySQL

Free tier : ✅

Nom : inowdb

User : admin

Password : inow2024

VPC : celui créé précédemment

Subnet : privé

Autoriser l’accès depuis EC2 via le SG

💡 *Correction CLI :*

```
aws rds create-db-subnet-group \
  --db-subnet-group-name my-private-subnet-group \
  --db-subnet-group-description "Private subnets for RDS" \
  --subnet-ids subnet-0a111111111111111,subnet-0b222222222222222
aws rds create-db-instance \
  --db-instance-identifier inowdb \
  --db-instance-class db.t3.micro \
  --engine mysql \
  --master-username admin \
  --master-user-password inow2024 \
  --allocated-storage 20 \
  --db-subnet-group-name my-private-subnet-group \
  --vpc-security-group-ids <sg-id> \
```

TP Global – Déploiement d'une architecture web 3-tiers sur AWS

6. Supervision & Coûts

- **Étape 6.1 – CloudWatch**

Console → CloudWatch → Metrics → EC2 → CPUUtilization

Créer une alarme : si CPU > 70%, notifier par email.

- **Étape 6.2 – Cost Explorer**

Activer **Cost Explorer**

Estimer le coût du projet via **Pricing Calculator**

7. Nettoyage

```
aws ec2 terminate-instances --instance-ids <id_instance>
aws s3 rb s3://inow-store-<votre-nom> --force
aws rds delete-db-instance --db-instance-identifier inowdb --skip-final-snapshot
aws ec2 delete-security-group --group-name inow-sg
aws ec2 delete-vpc --vpc-id <id_vpc>
```